

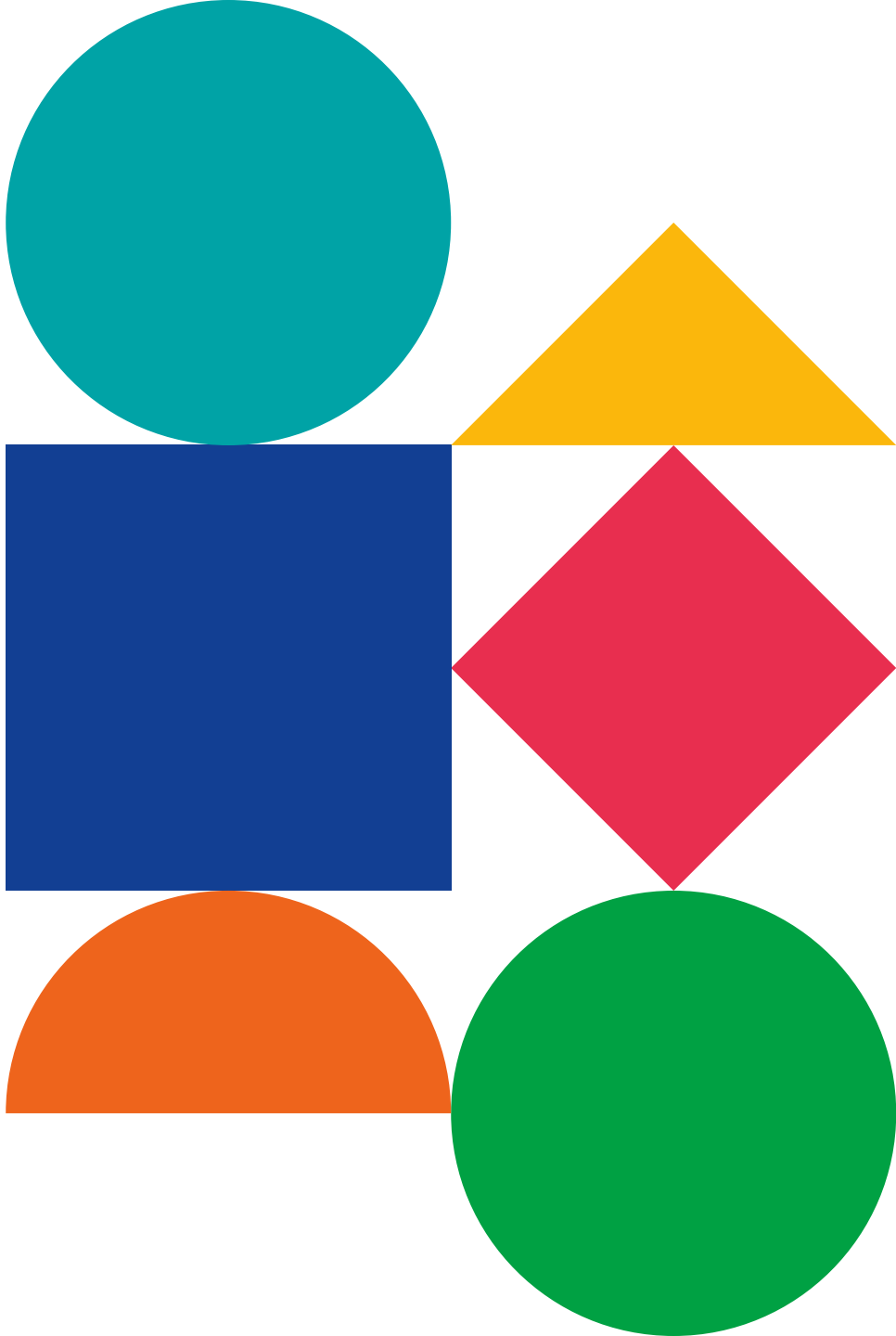
Could you install it all ?

Versioning with Git & GitHub

For code and documentation

Bruna Piereck

April 2026



Uses case

- Covered in the course
  - Single user
  - Collaborating

Back up

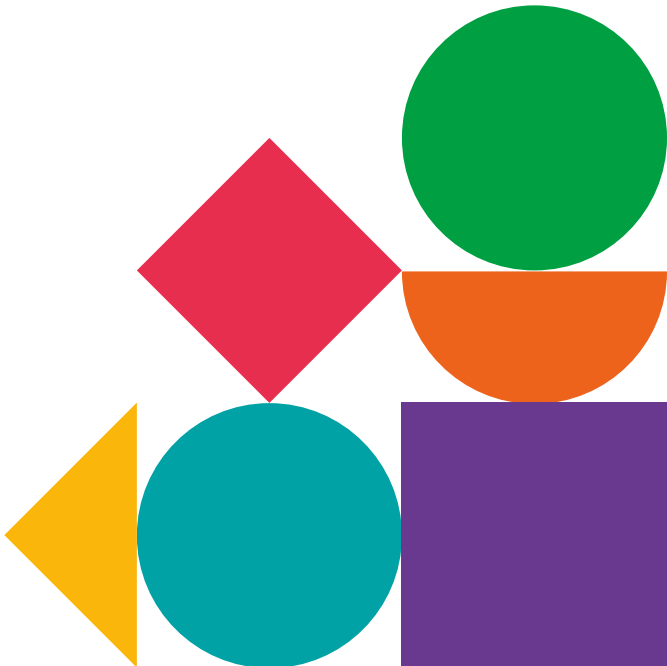
History

Recover previous state

Working remotly

Publication

- Not to be covered
  - Contribute to open-source project

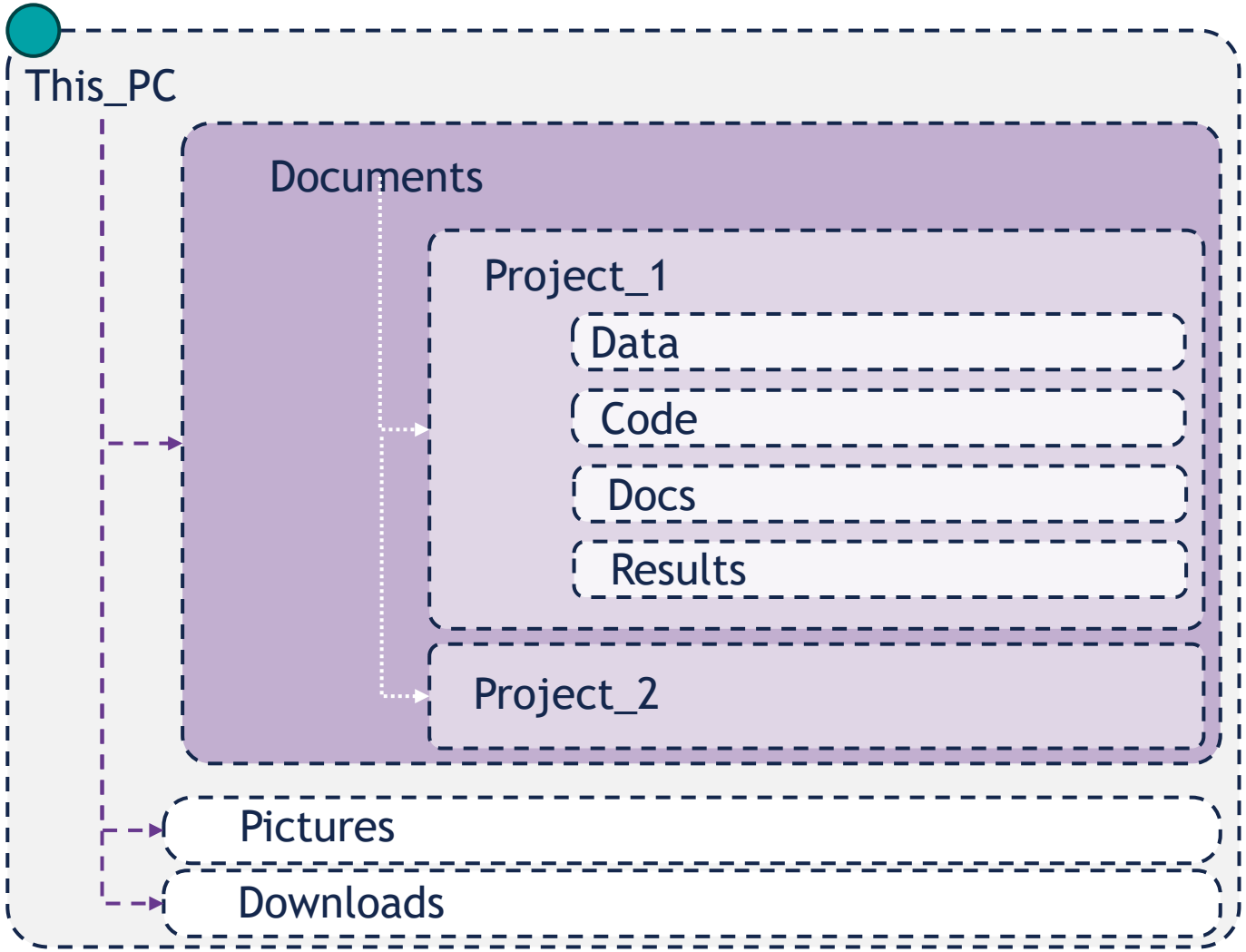


Folder structure

- It is hierarchical
  - Folder appear bold/highlighted
  - Files not

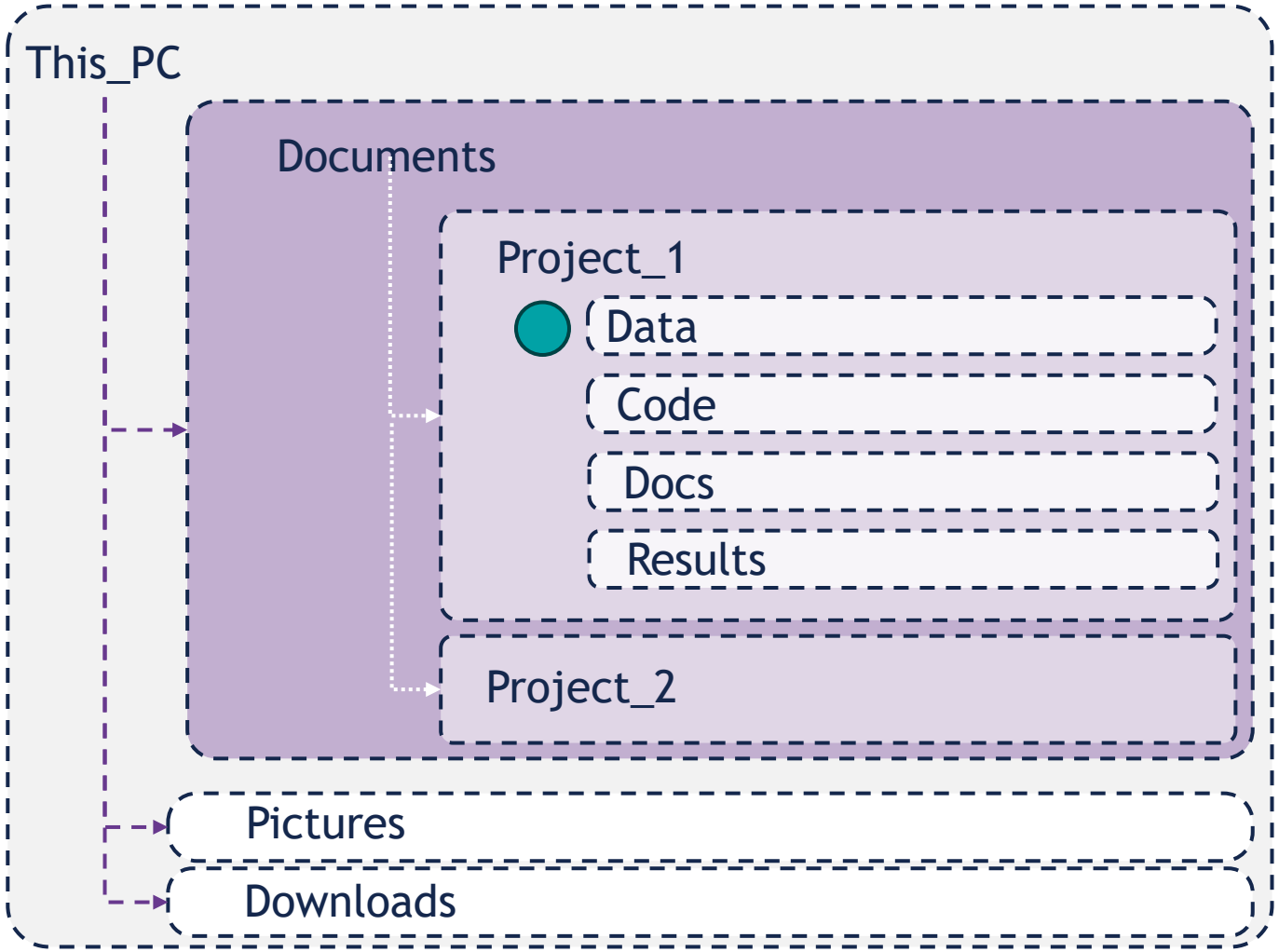
```
Jan 23 15:11 'Blocos de Anotações do OneNote'/
Jan 23 15:42  Copilot/
Feb  2 09:45  Courses/
Jan 23 15:11 'Custom Office Templates'/
Sep  5  2024  Default.rdp
Jan 29 11:34  Dell/
Apr 13 09:33  Meeting_minutes/
```

```
Apr 17 14:09 CITATION.cff
Apr 17 14:09 LICENSE
Apr 17 14:09 README.md
Apr 17 14:10 data
Apr 17 14:10 docs
Apr 17 14:10 exercises
```



UNIX basics review

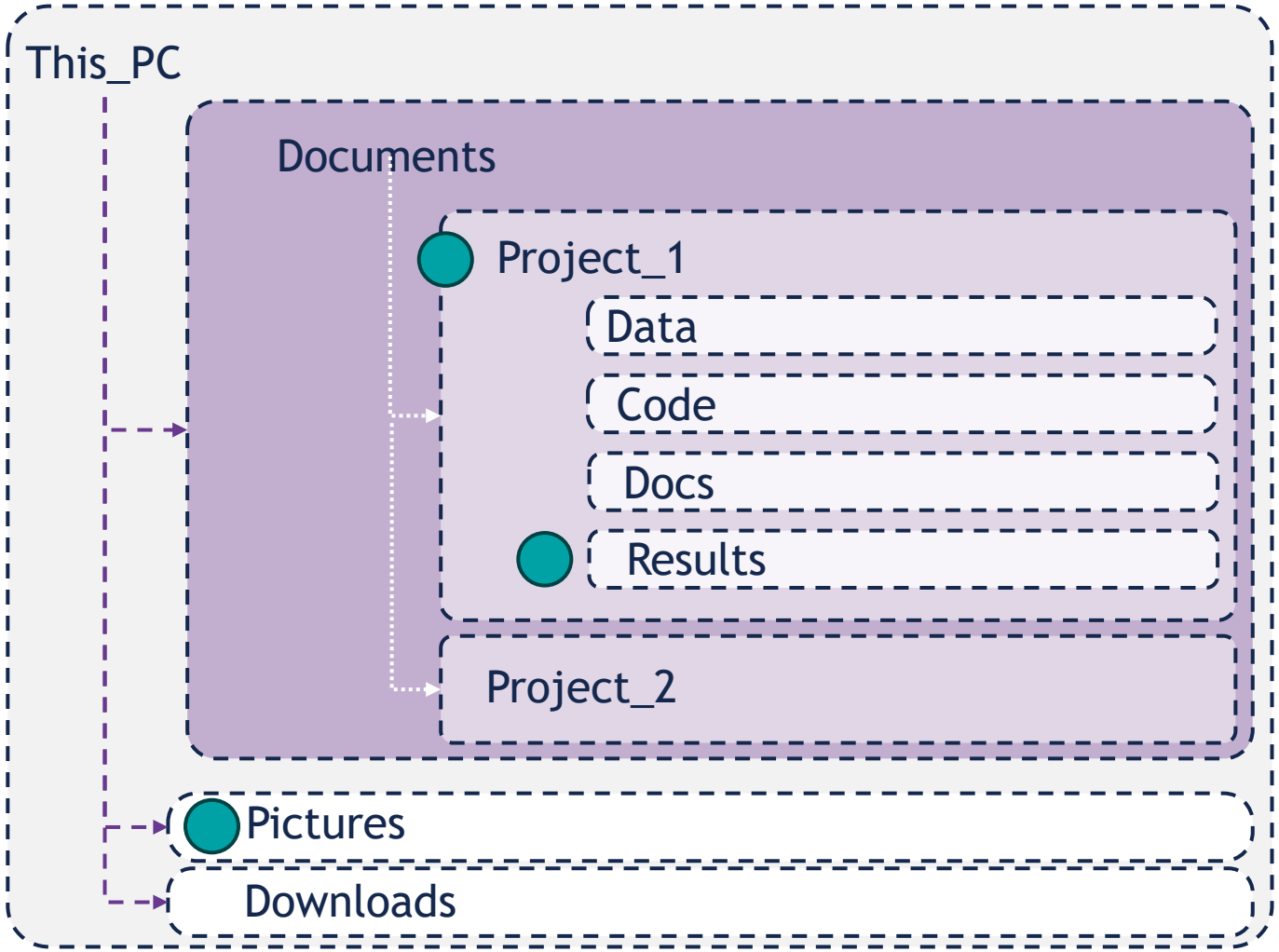
- How to?
  - Find the path to directory Data?



P.S.: Use <tab> to auto-complete

UNIX basics review

- How to?
  - List all content from directory Project_1?
  - Move from directory Project_1 to directory Results ?
  - Move from directory Results to directory Pictures ?



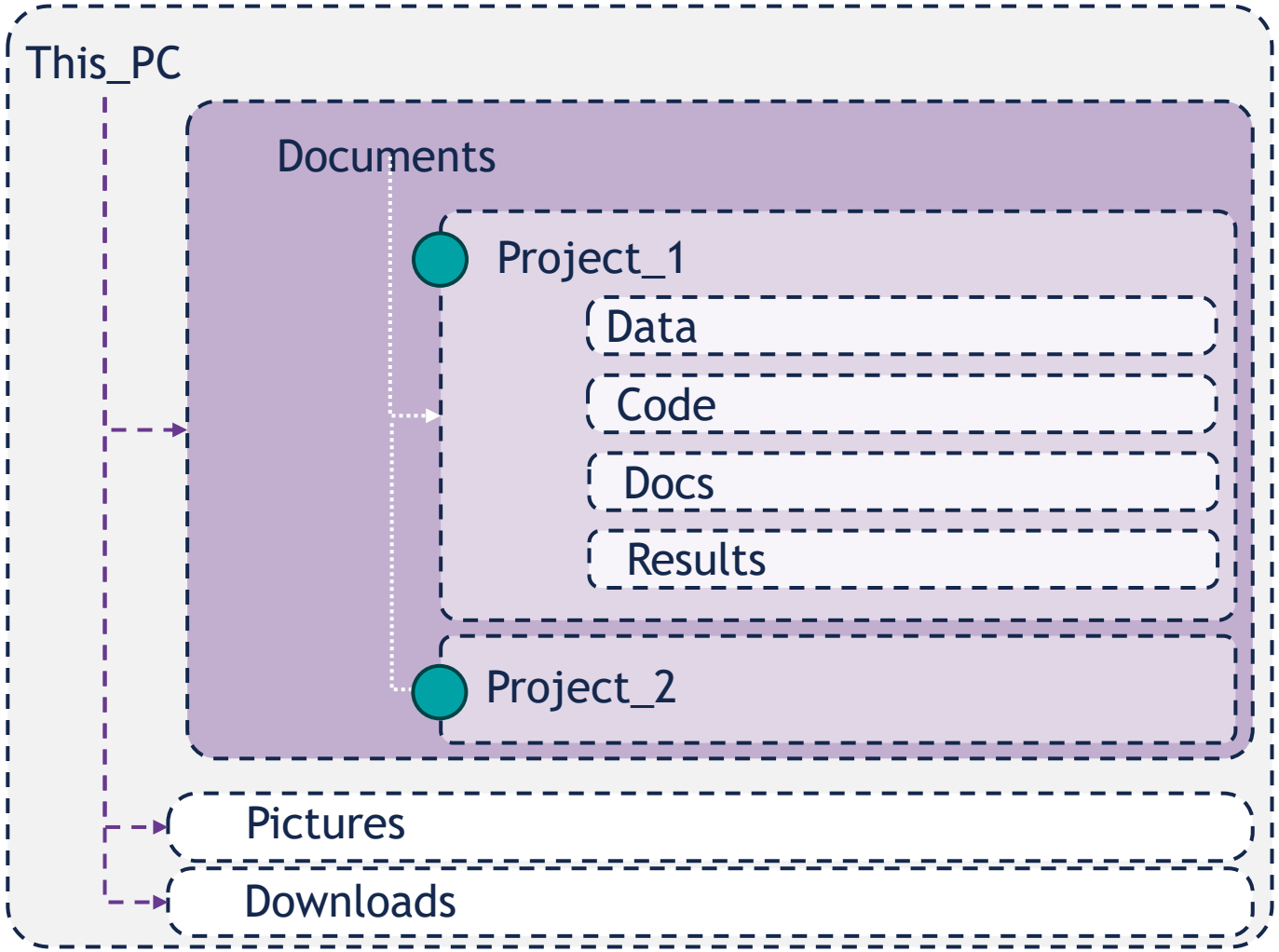
UNIX basics review

- Why ?
  - Why are the directories named

    Project_1 | Project_2

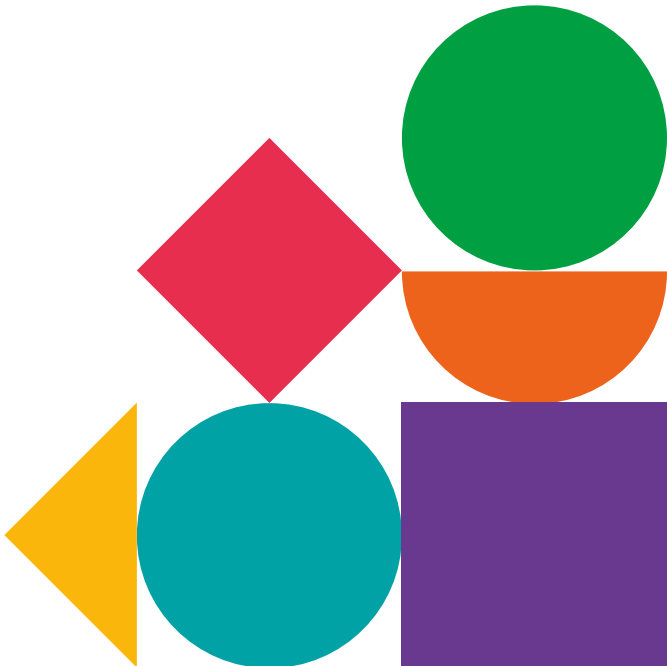
    Instead of:

    - Project one | Project two
    - Project 1 | Project 2





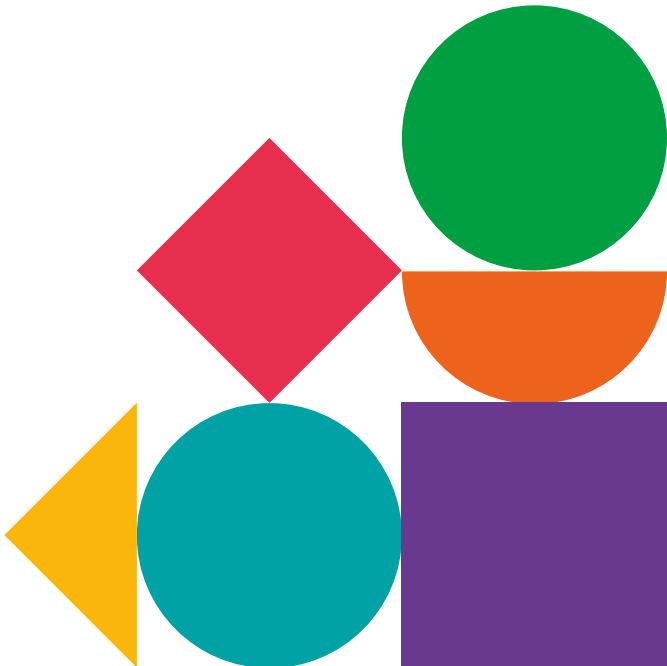
Have you been in this situation?



Class Project

- Goal

1. Creat and update a DICTIONARY of concepts
2. Creat and update a CHEATSHEET with all the commands
3. Make it all available in GitHub with version control



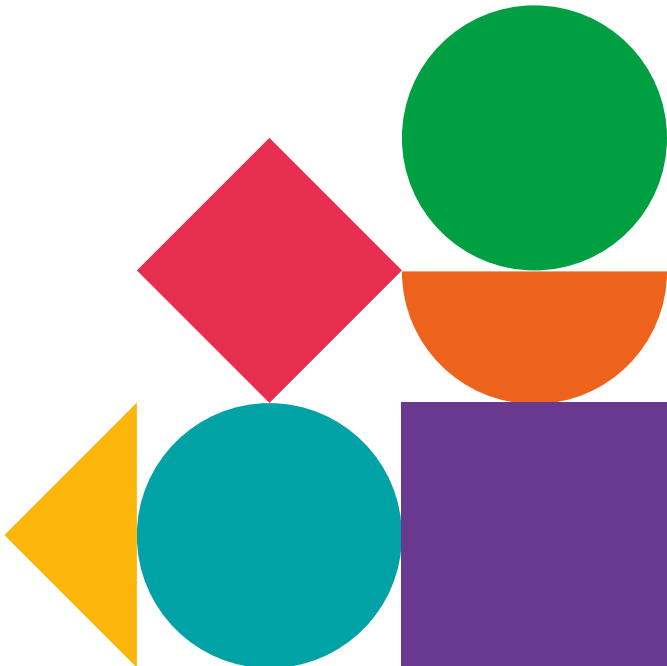
Class Project

- Starting the project

1. Creat a folder where your project is going to be developed
   - Make sure is not inside a git project
     (If you ever used git before)

2. Only files in text format should be created
   - MarkDown is preferable





MarkDown

- Code for text formating
  - # = Title
  - ## = Subtitle
  - ### = Sub- Subtitle

  Hierachi

  - *Bold*
  - **Italic**

  - ``` code ```

  - 1. = numeric topic
  - - or * = list topic

Code view

```
# Title

## Subtitle

- Topic

- Topic *very important*

- Topic **in italic**

```my code```

```

My

Multiline

code

```
```

Formated view

Title

Subtitle

- Topic
- Topic **very important**
- Topic *in italic*

```
my code
```

```
My
Multiline
code
```
```



What is happening?

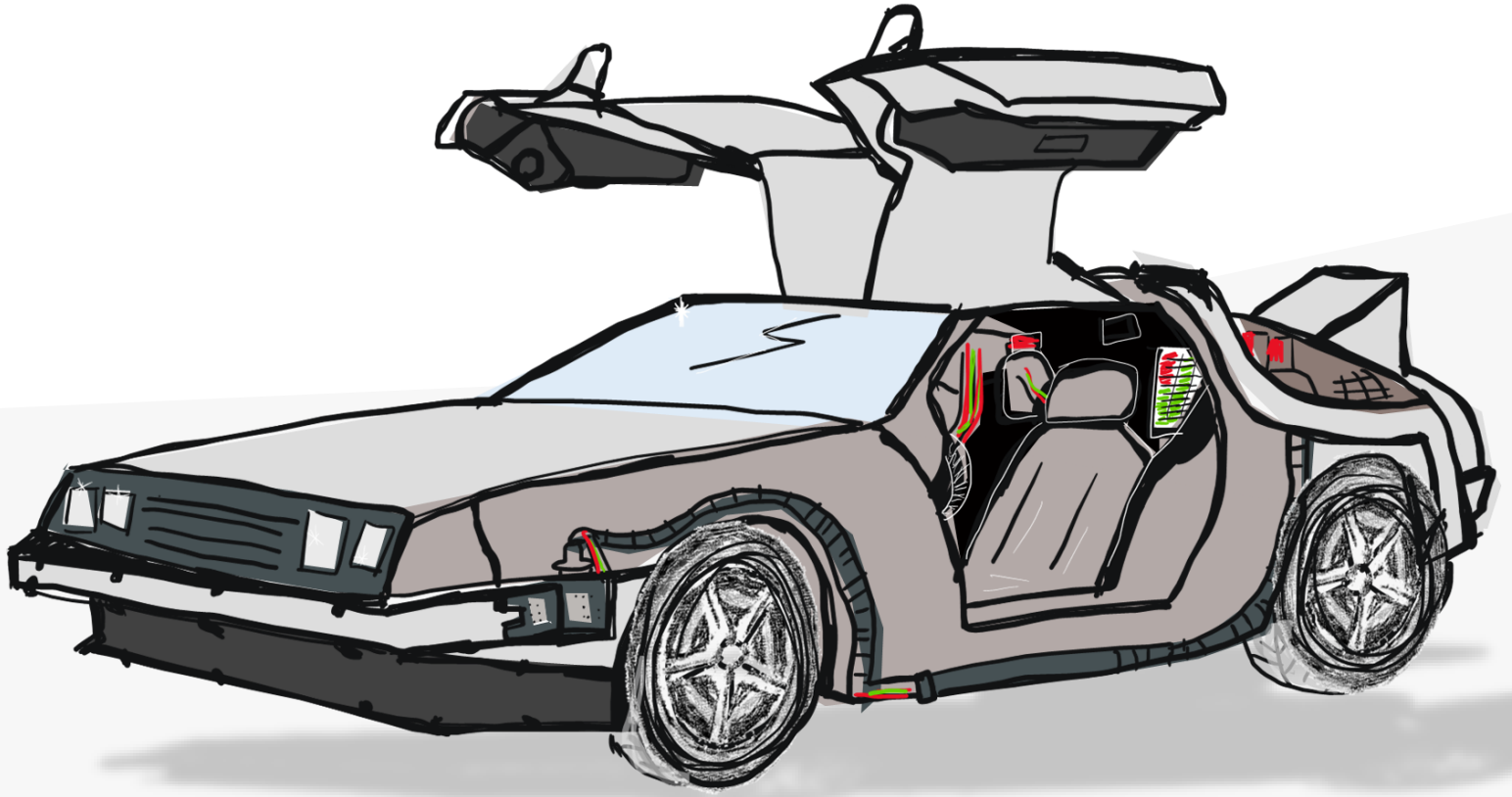
How would you solve?



Think of a time machine

(like in *Back to the Future*)

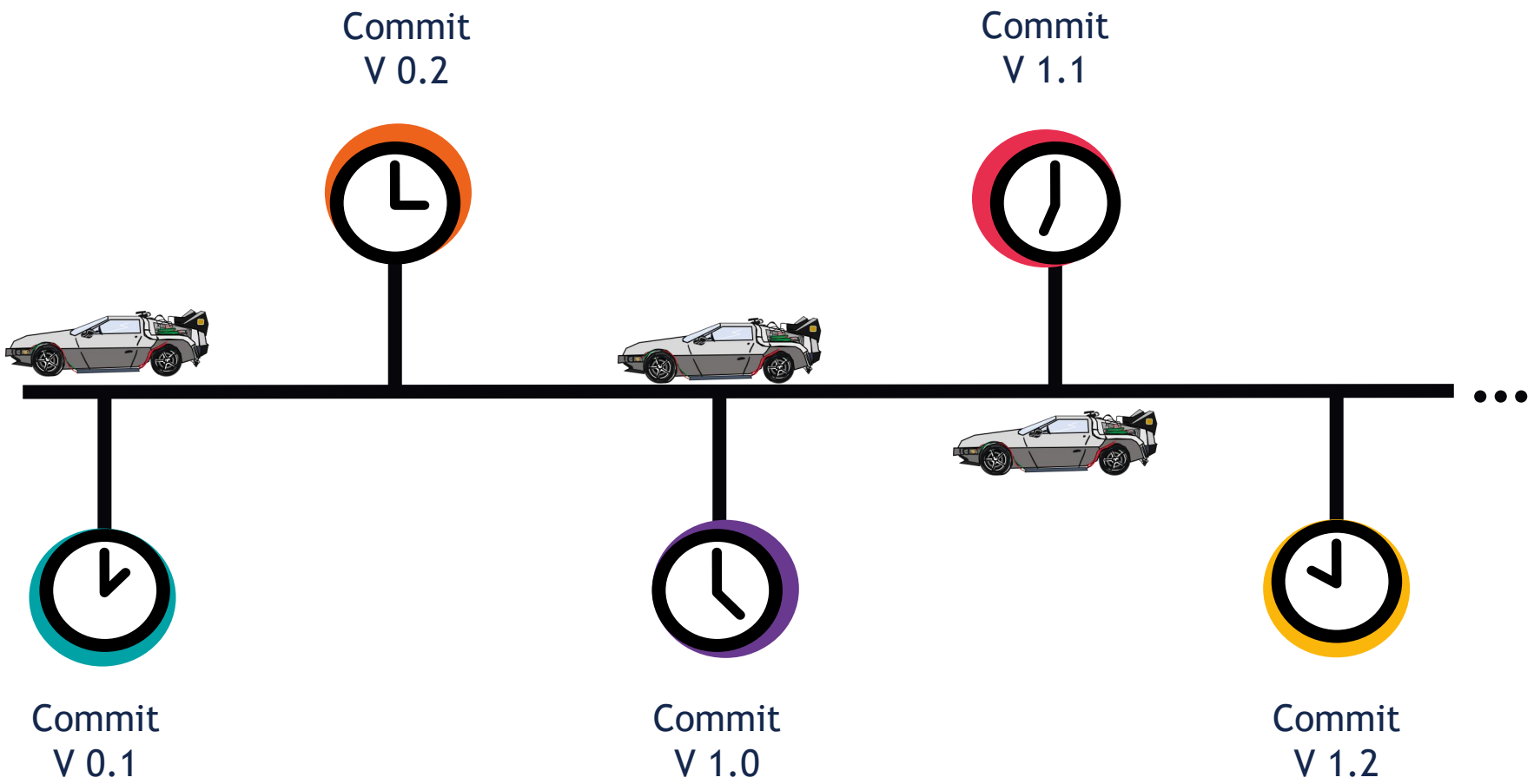




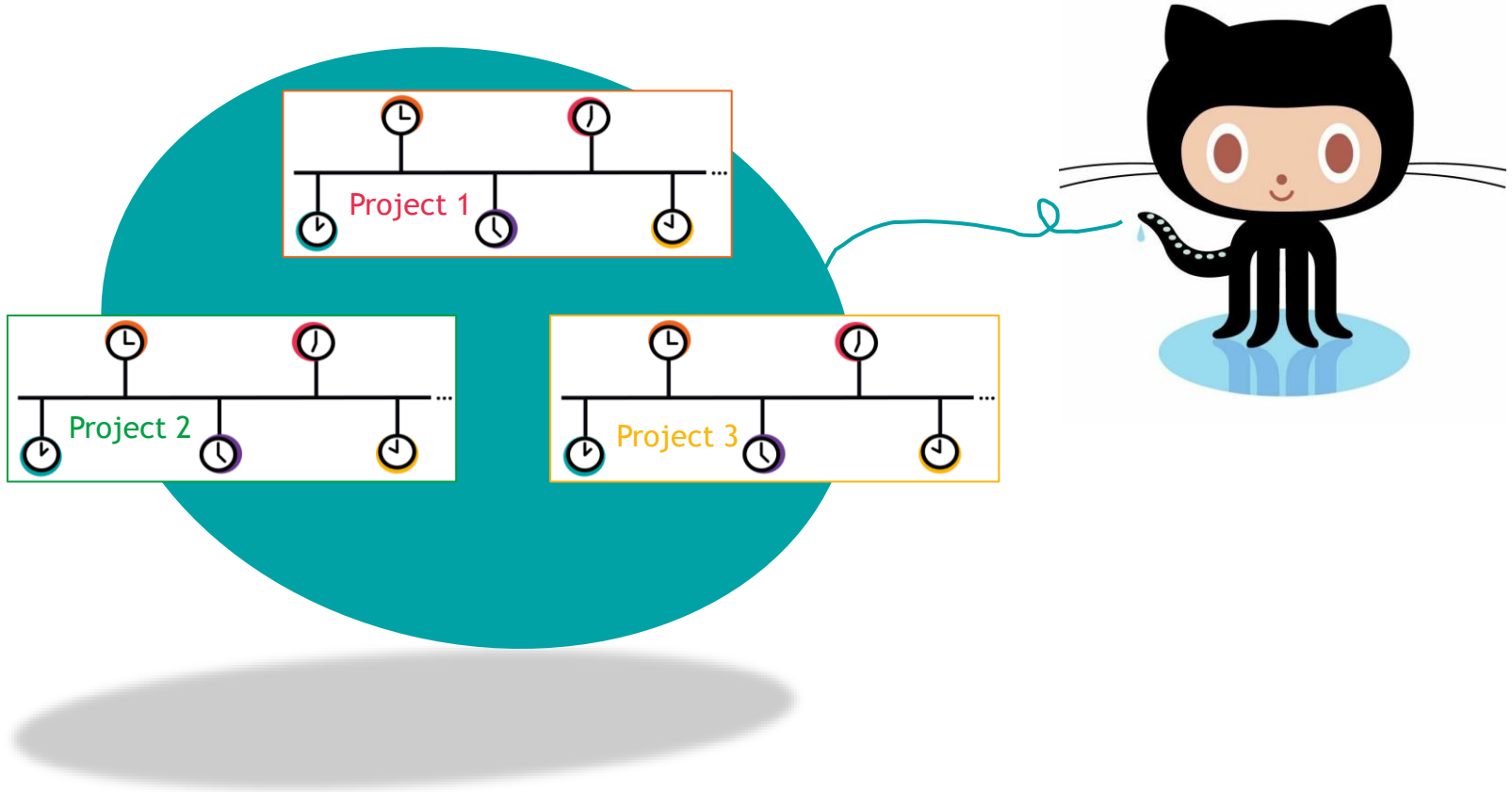
differences

git *vs* GitHub

Git repository = Timeline



GitHub= Timeline Backup



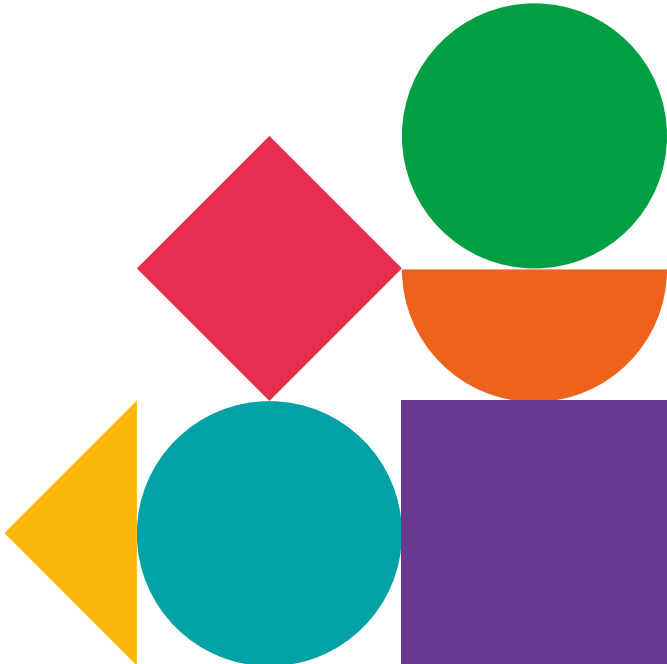
ACTIVITY

1. Create a concepts_dictionary.md file
   - Write your first concepts
     - What is a git repository?
     - What is GitHub?



VS





Starting a timeline

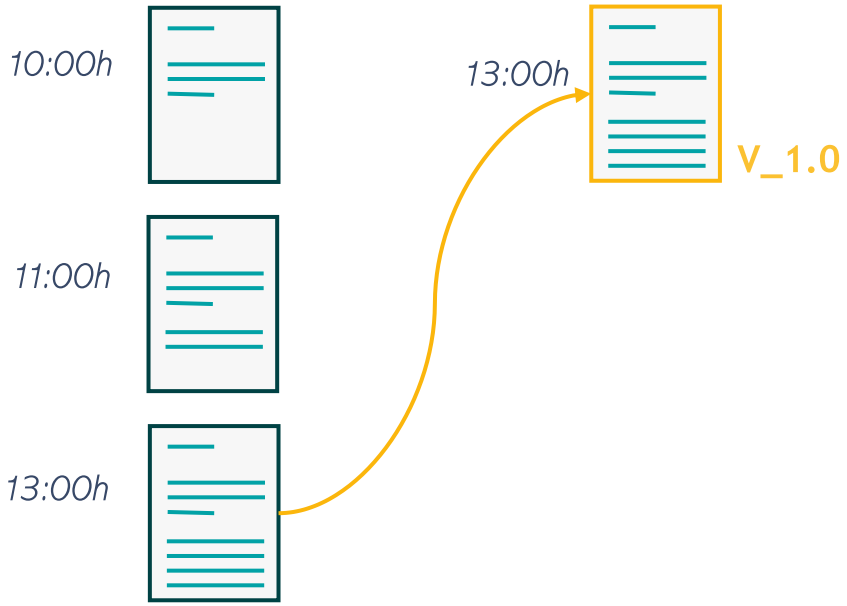




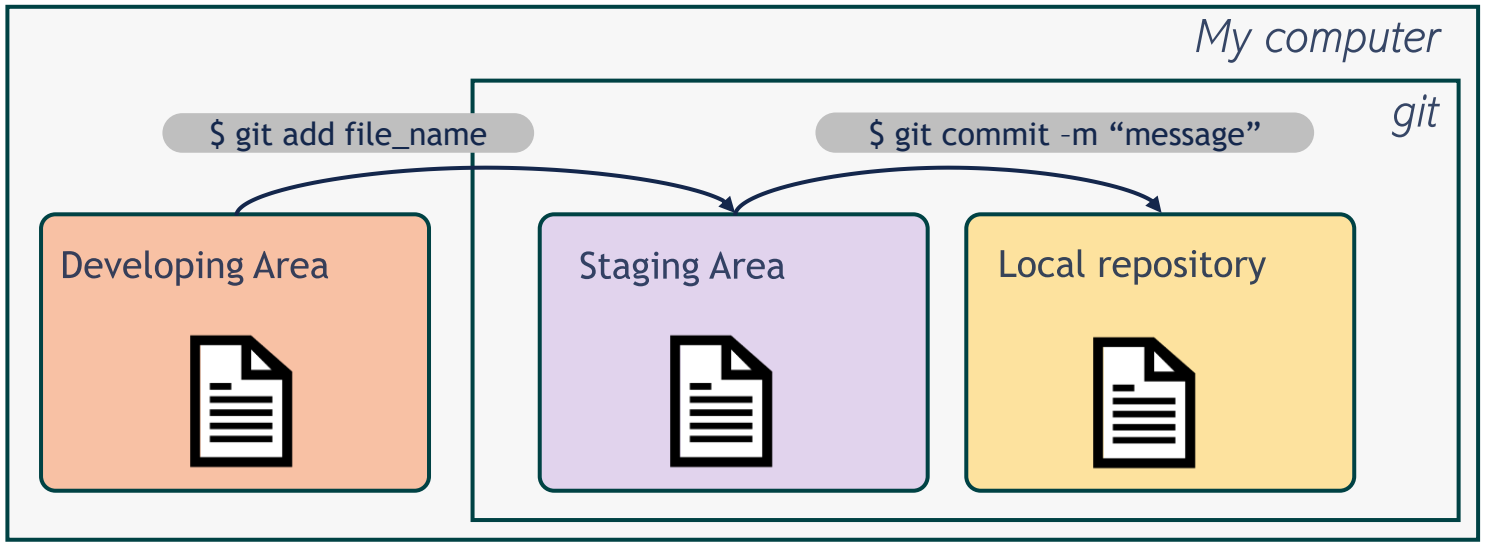
- Necessary to start versioning
- Next step: *"define"* versions in the timeline

Defining **one version**

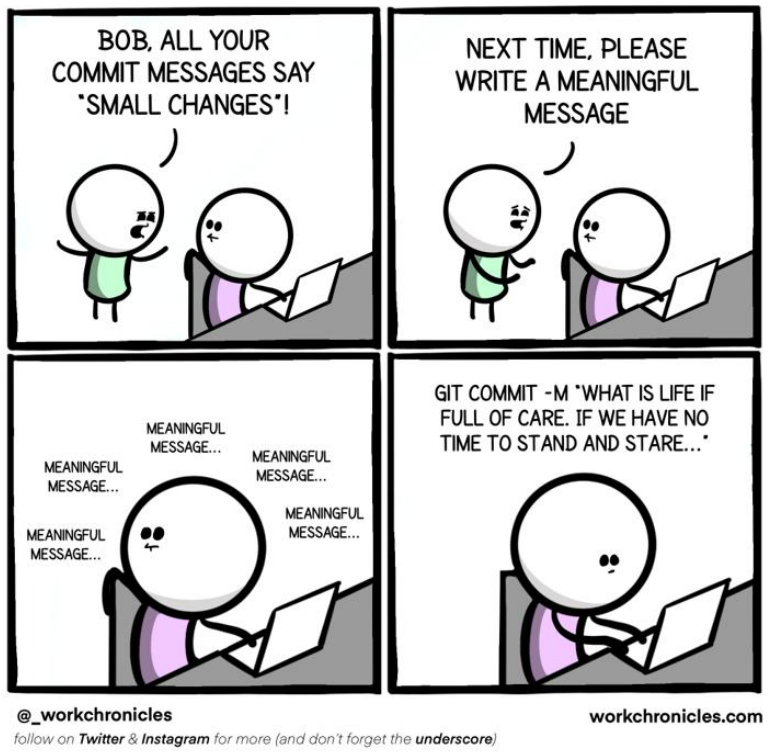
- Save changes ≠ Saving a commit (version)



Routine to *save/define* a version



Commit message



user$ git commit -m "*meaningful message*"

- What is meaningful?
  - Why was changed
  - How this addresses the issue
  - Effects due the change
  - Limitations of the change

Be descriptive !!!



QUIZ

1. Which message better suits a commit to update a hypothetical project description ?

   a) Update TUTORIAL.md file

   b) Added ‘This tutorial is for beginners’ to TUTORIAL.md

   c) Disambiguate this project from **Project X** by stating the audience

- What is meaningful?
  - Why was changed
  - How this addresses the issue
  - Effects due the change
  - Limitations of the change



Before we start

git configurations

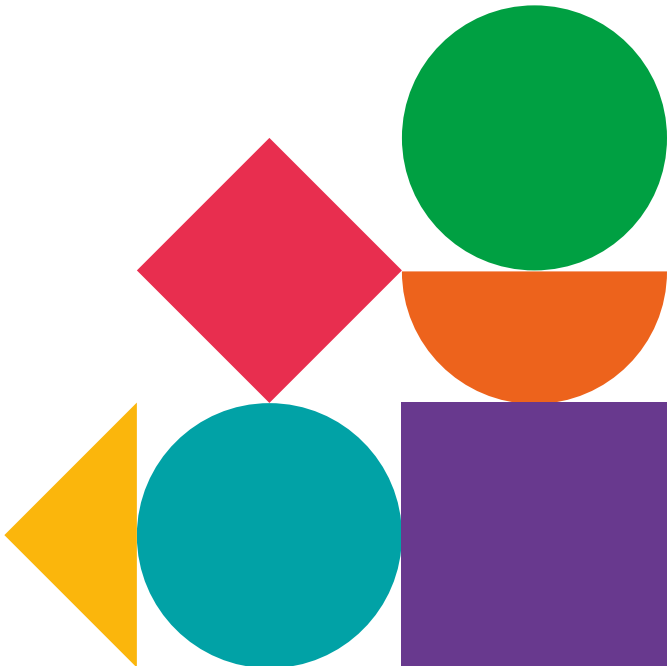
Check if git has your information in the config file

```
user$ git config --global --list
```

Add or change your information

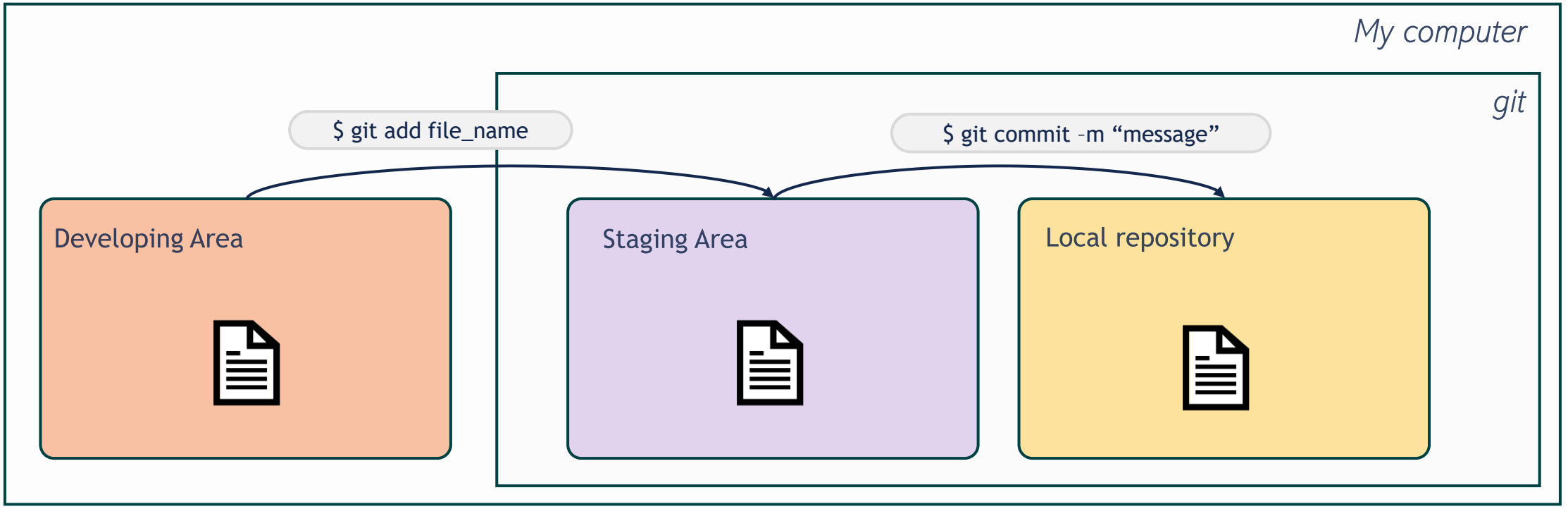
```
user$ git config --global user.name “Your Name”
```

```
user$ git config --global user.email “your_email@domain.com”
```



Save a point in time: one version

Routine to *save/define* a version



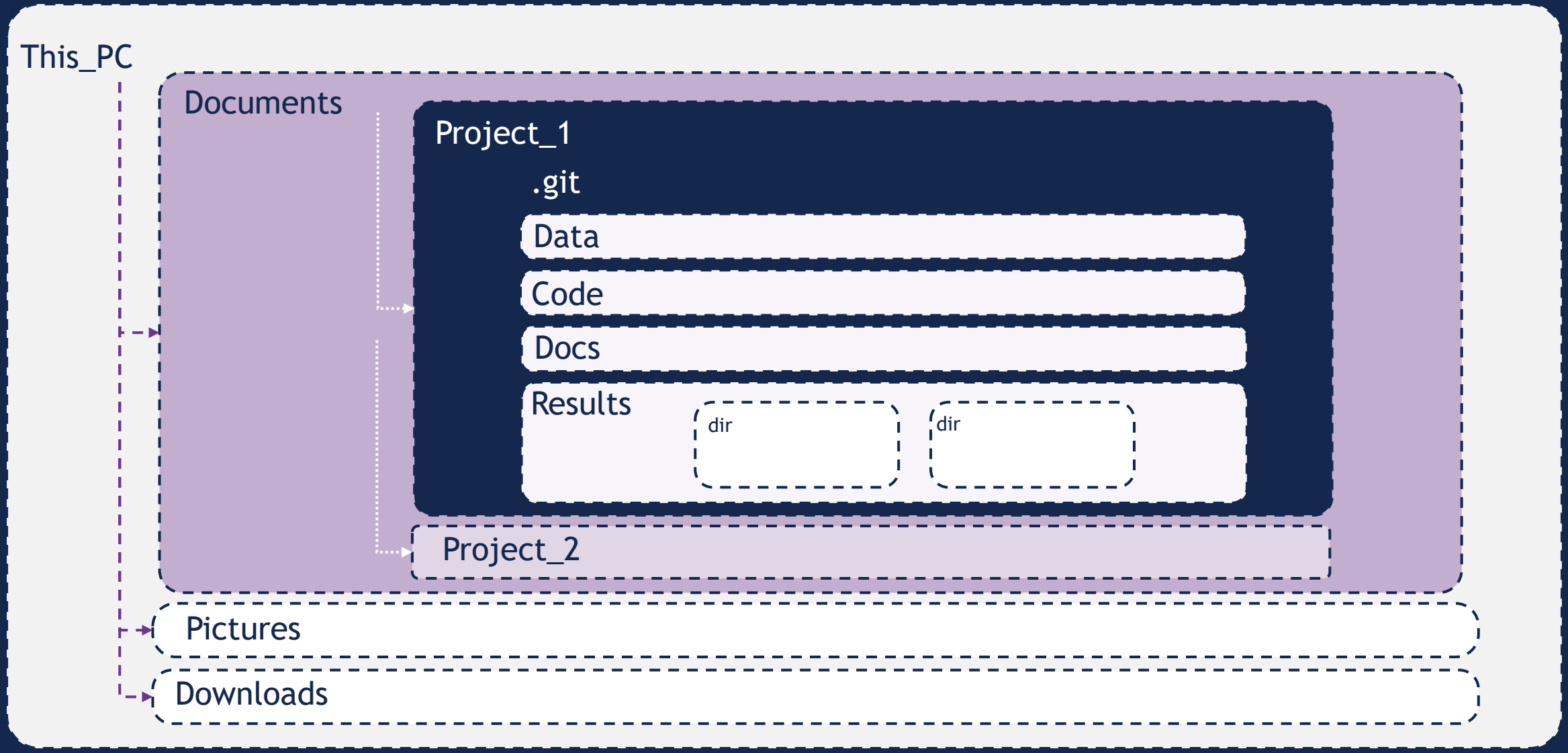
QUIZ

1. What if we create a subfolder of the main project. Is this folder under git version control? Do we need to initialize git again inside of the subfolder?

   a) No. We need to “git init” again.
   b) No. We need to work in one folder only.
   c) Yes. The software will track it all.
   d) Yes. But we need to do “git init again”



Explaining


This_PC
Documents
Project_1
.git
Data
Code
Docs
Results
dir
dir
Project_2
Pictures
Downloads

Conceptual areas

Where is the file?

Where is GitHub ?



Where is git ?

1. Development Area

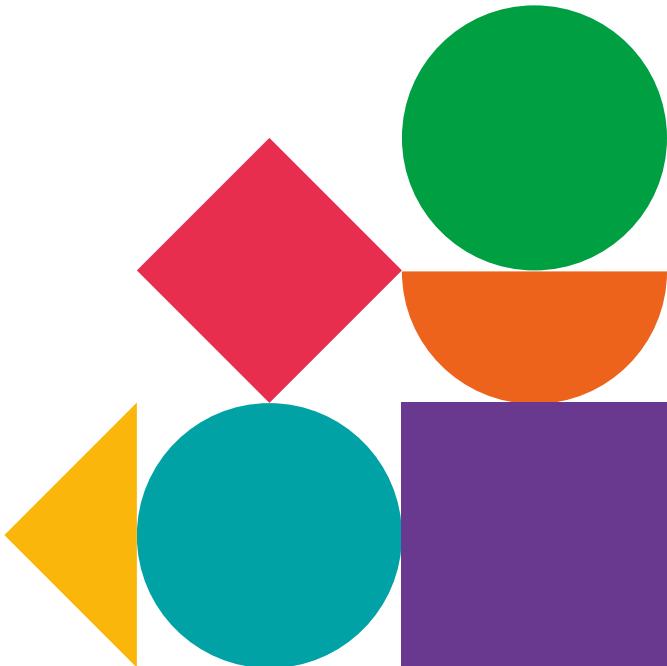
- Project folder
  - Where files & sub-directories are managed



ACTIVITY

2. Go to your development area and add this new concept in your dictionary.

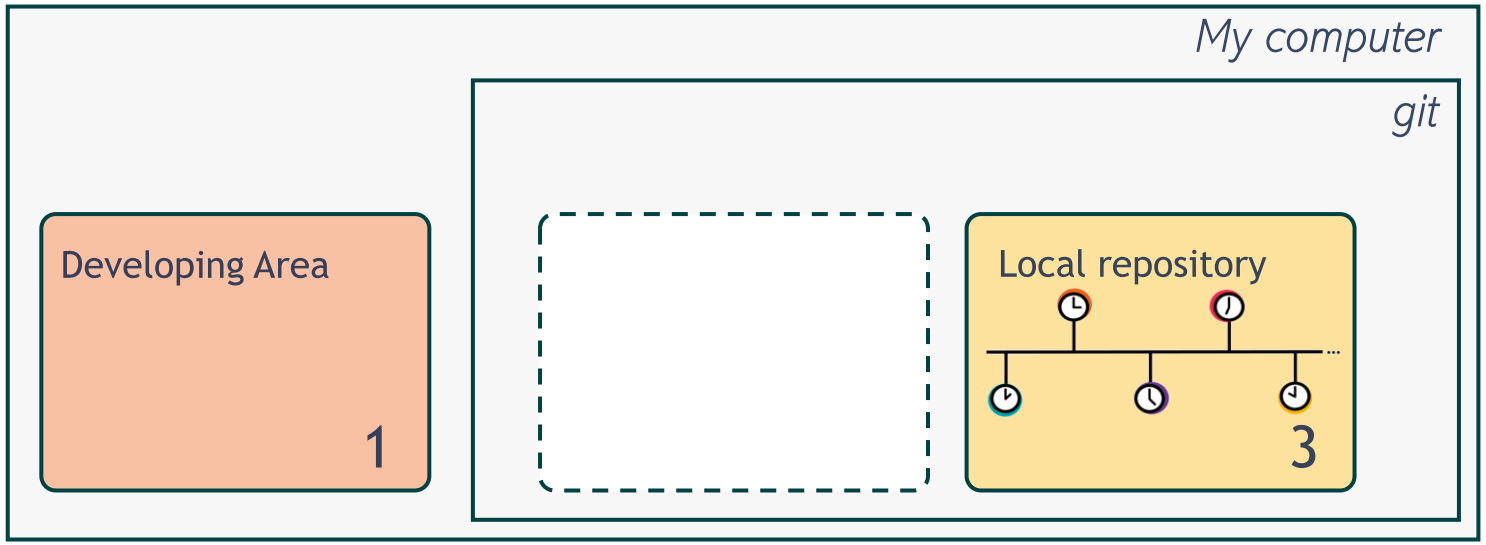
   - Once ready `git add file` > `git commit -m “meaningful message”`



3. Saving the TIMELINE (version history)

- git local repository
  - Keep your versions
  - Create points in time (timeline)

$ git commit –m “message”



QUIZ

- Why do we write a commit message?

This message will help our collaborators and ourselves in the future to understand why changes were made. How the issue was addresses and if this the solution had any impacts or limitations.



QUIZ

- What happened if we forget about the “-m” when committing a file?

  `git commit`

  It opens an editor and allows you to write longer, and more detailed messages organized in paragraphs.

  - Choose your editor

    VSCode: `user$ git config --global core.editor “code --wait”`

    Nano: `user$ git config --global core.editor "nano“`

    Set default (vi): `user$ git config --global --unset-all core.editor`

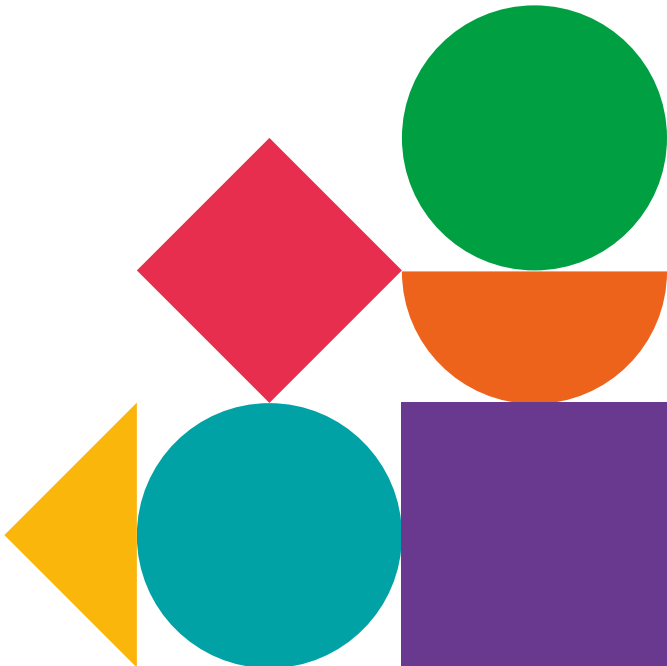


ACTIVITY

3. Go to your development area and add this new concept in your dictionary.

   - Once ready `git add file` > `git commit`

   - Commit with a long message using the editor you chose before



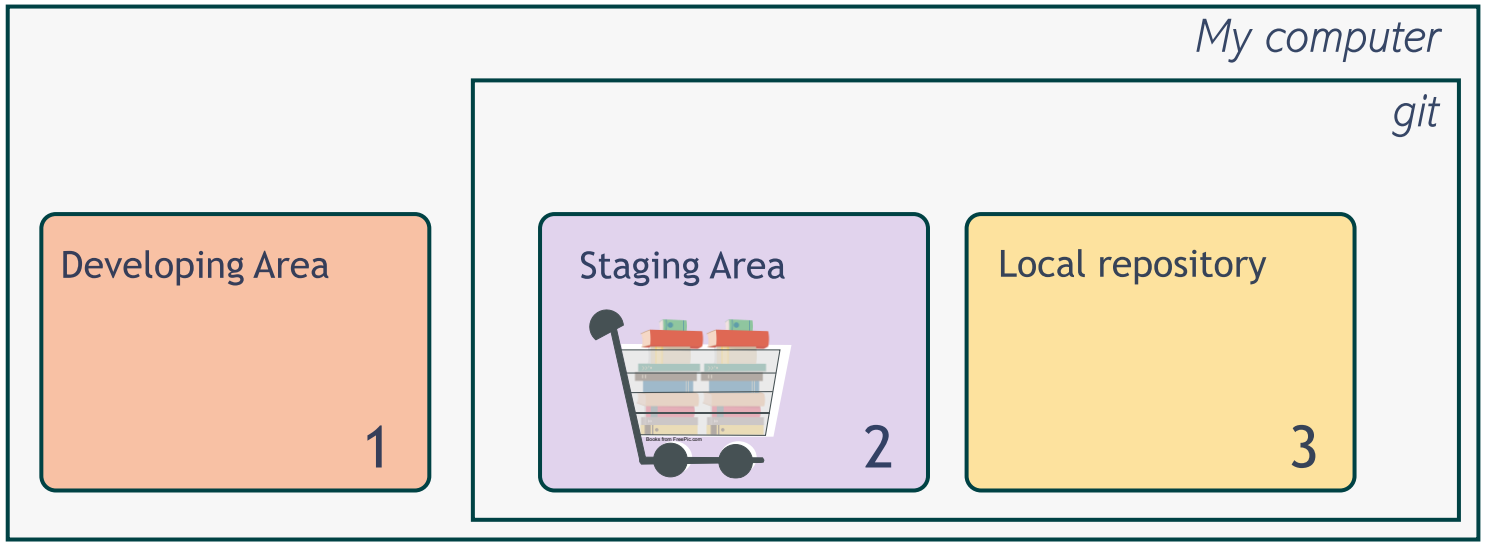
staging
2nd area

before saving

2. Organize before moving to a permanent location

- A storehouse
  - Organize the snapshot
  - Useful with multiple files

$ git add file_name.md



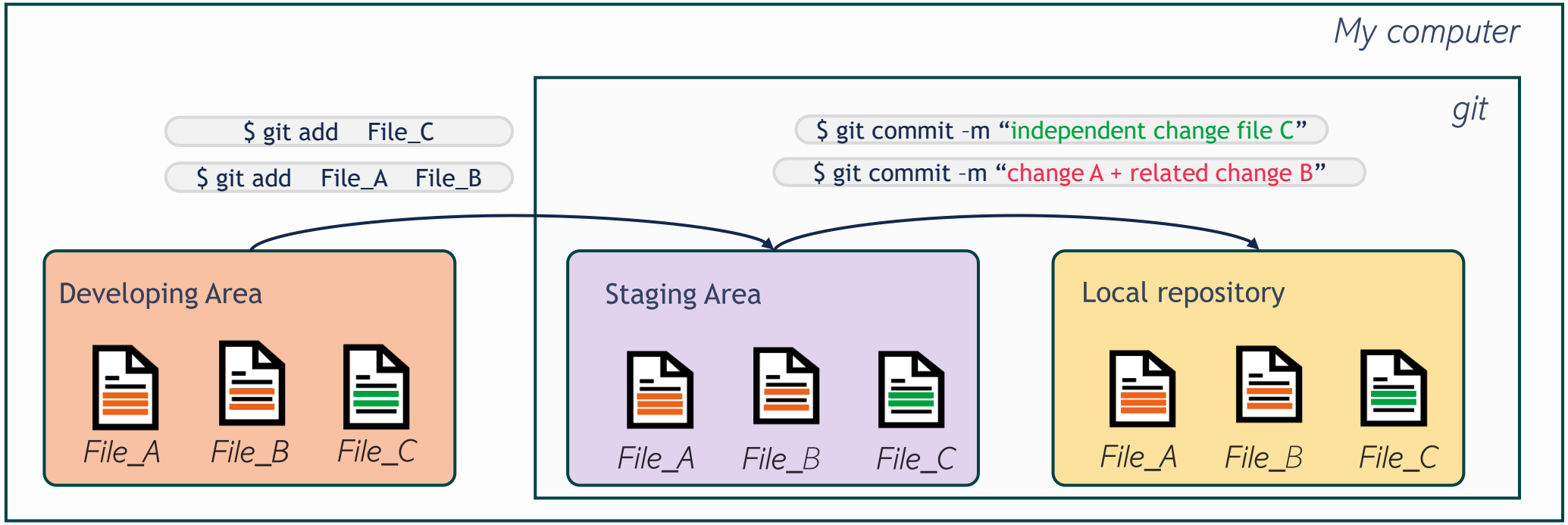
Why it is useful to stage files?

- Several files and folders
- Organize of your timeline in clear units
- Think critically of your commit messages






Staging before saving





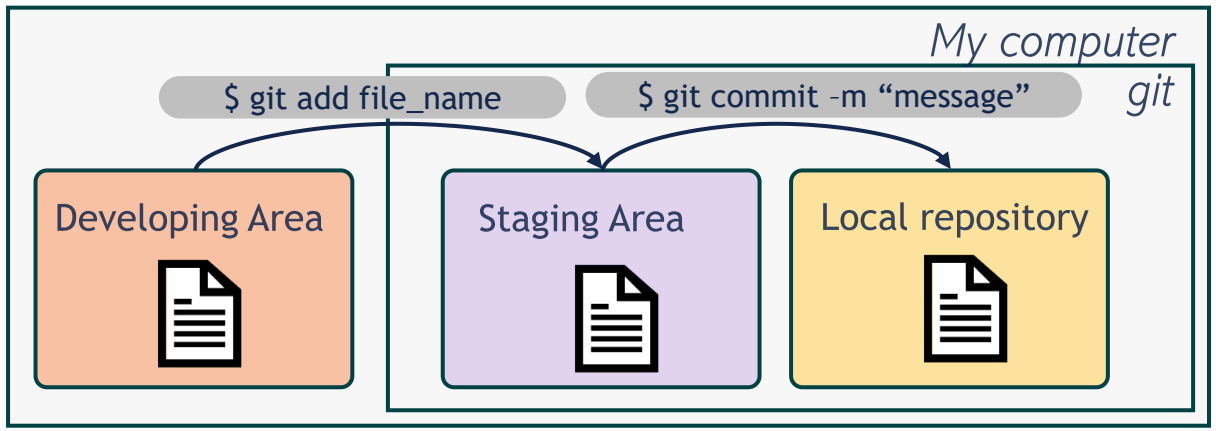
ACTIVITY

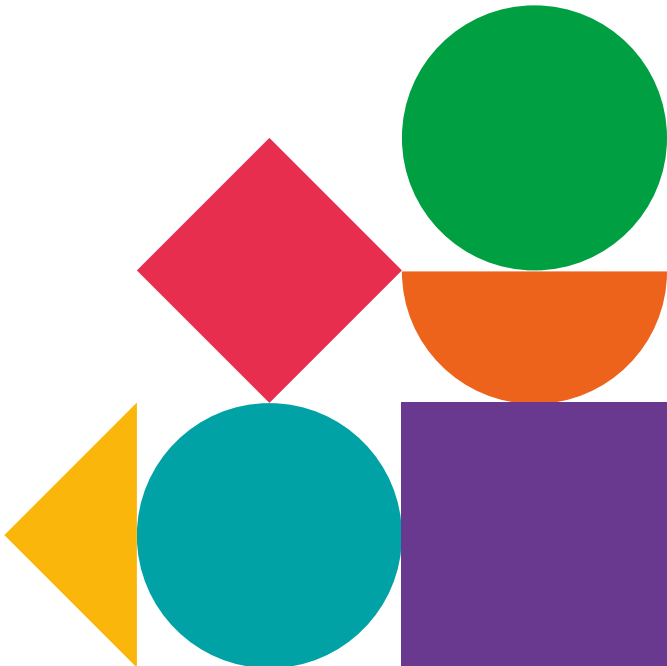


Do not commit anything YET

4. Go to your development area and

- Add the concept of using the Staging area to your dictionary
- Create another file: commands-git.md
- Add the basic routine to define/save a version of your project





ACTIVITY

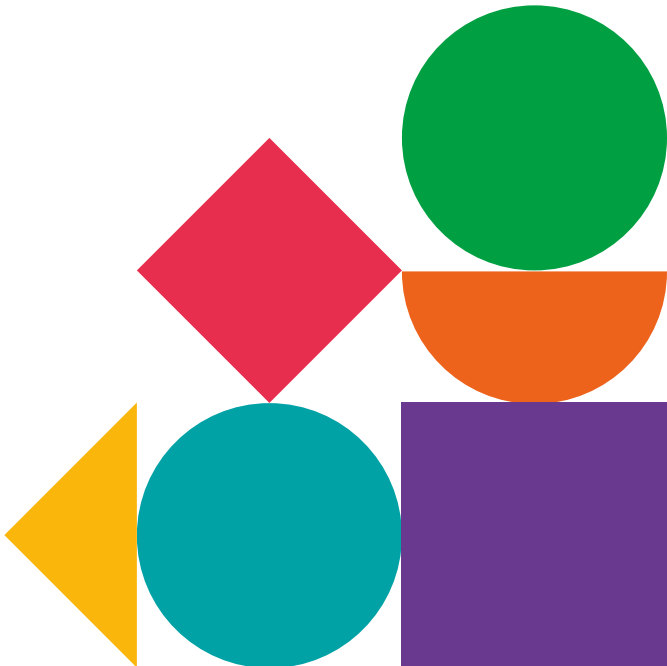


Do not commit anything YET

5. White on a paper
   - A commit message if each file was independently committed
   - A commit message if files were committed together



- Discussion time
- Committing time



- How does staging helps organizing the timeline?

  A. Staging all documents at once
     - Creates insufficient versions
     - Confuse and unclear history

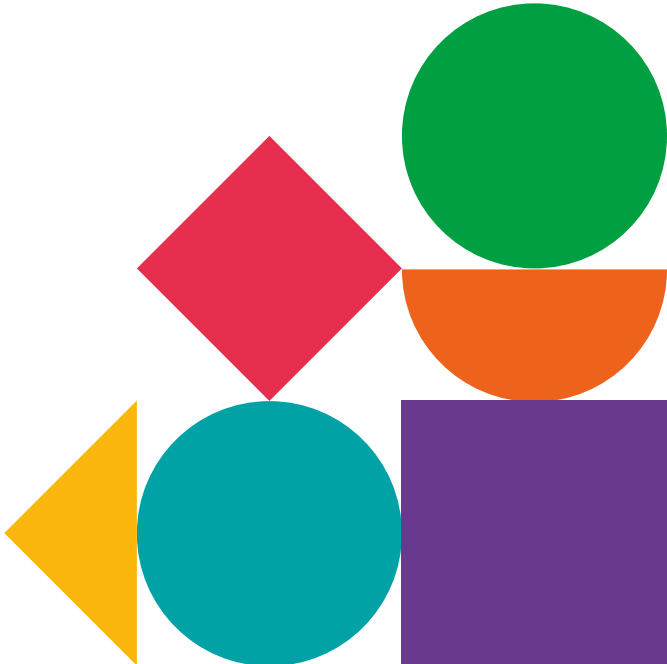
  B. Staging only one document at a time or
     Staging every few lines
     - Repetitive very long history
     - Difficult to navigate

Using it thoughtfully creates a logical history, easier to navigate. Like an organized library.

ACTIVITY

6. Add an image to the developing area
   - The image will be shared in GitHub later
   - It will be se by your collaborator and maybe others

CHOOSE WISELY



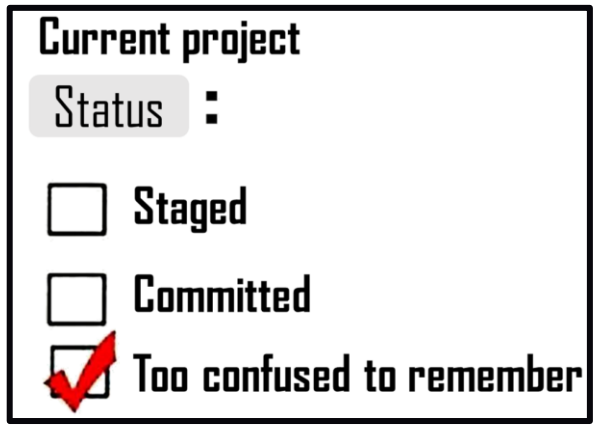


Coffee time

Where is the image?

Which conceptual area?

```
user$ git status
```



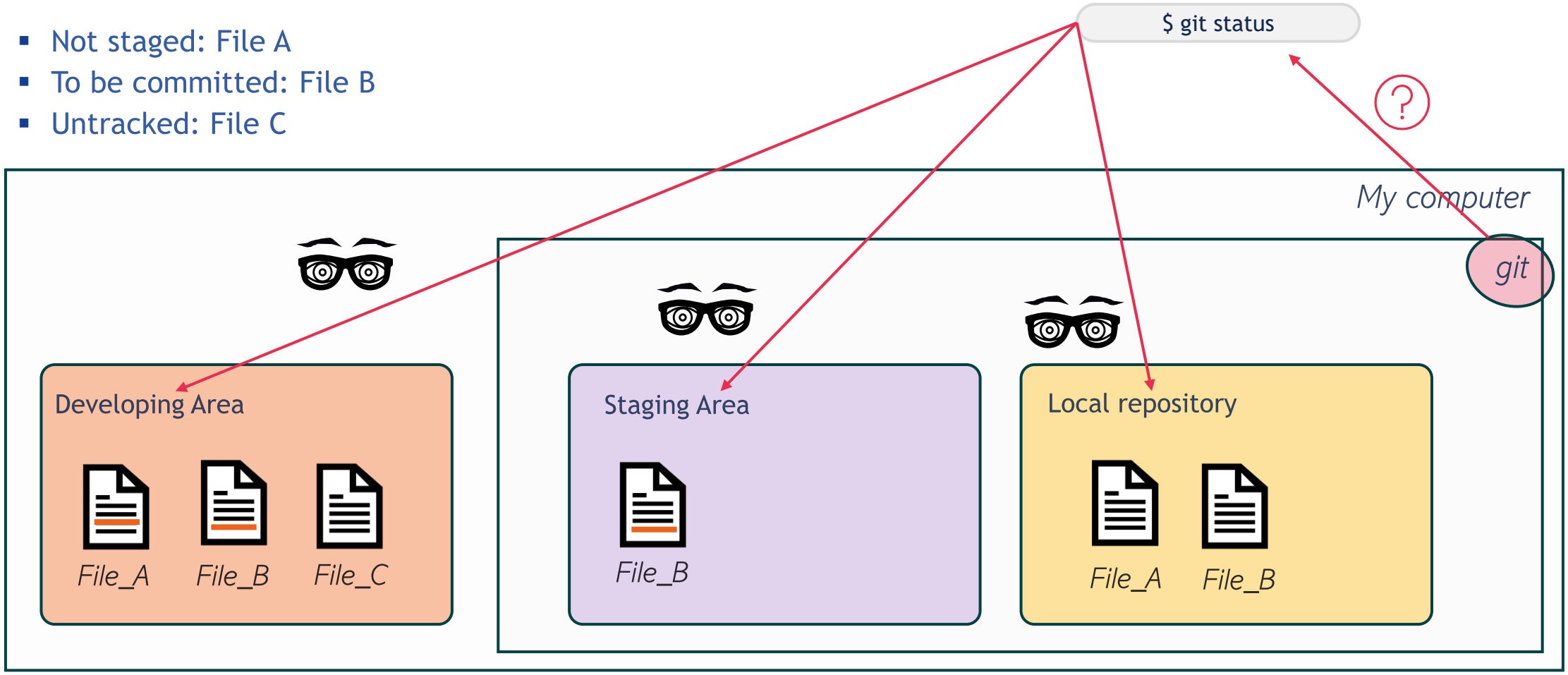
- Checks the status of each file
  - To be committed:
    New version in the staging area
  - Not staged:
    New version in the developing area
  - Untracked:
    Never managed by git



Always check: it will save you trouble

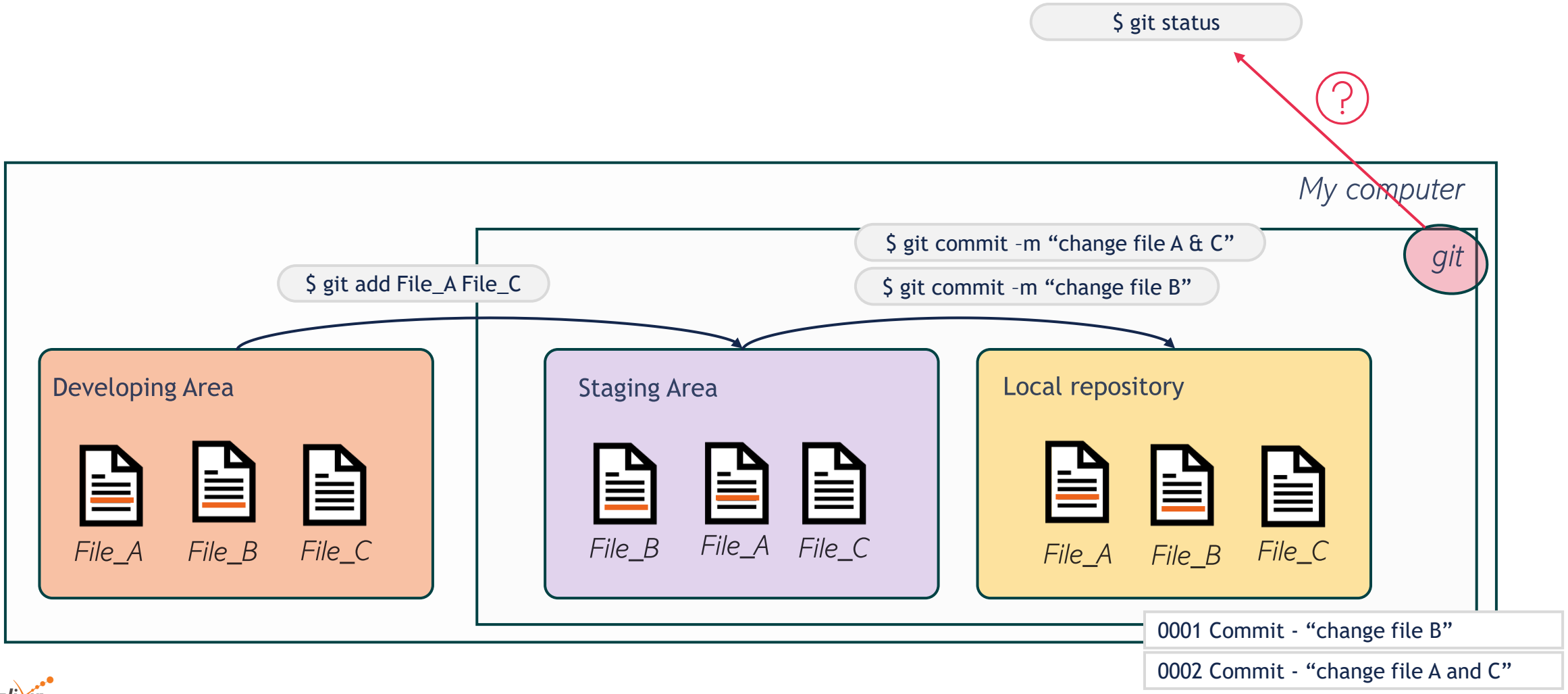
NEW Routine to *save/define* a version

- Not staged: File A
- To be committed: File B
- Untracked: File C



Always check: it will save you trouble

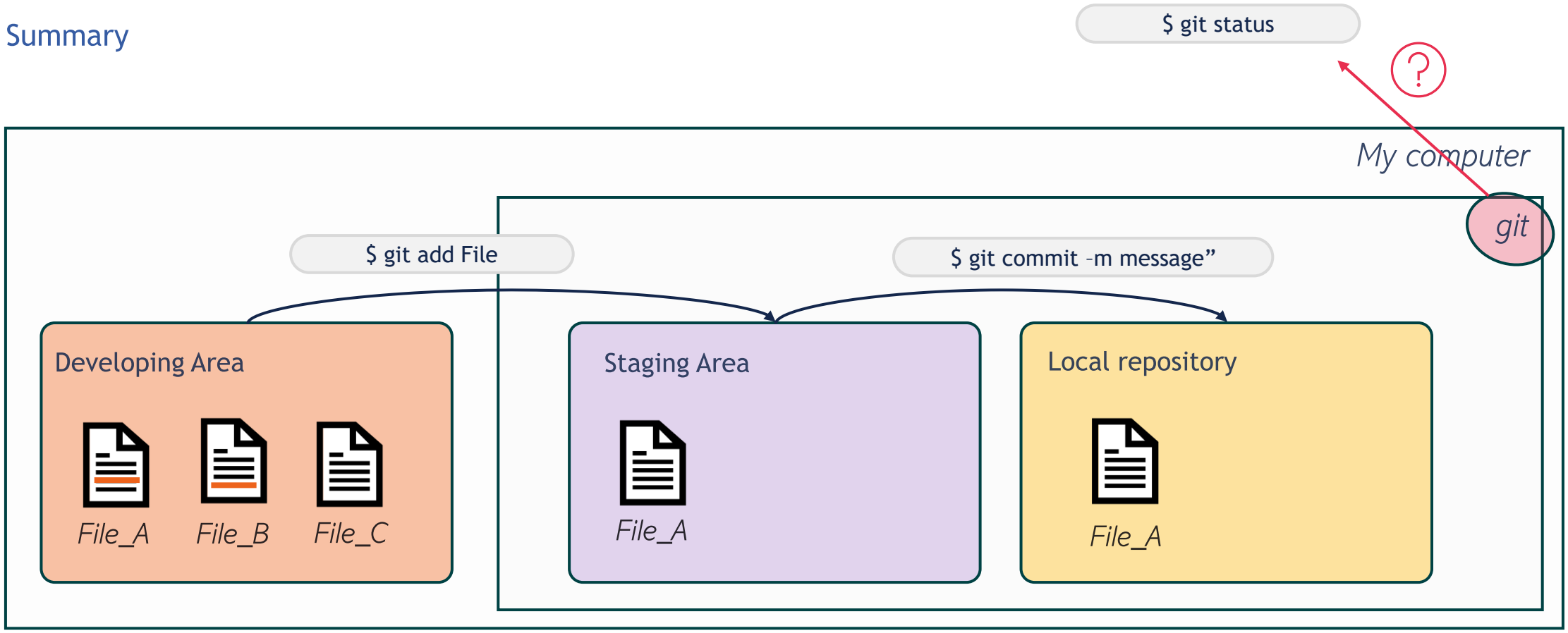
NEW Routine to *save/define* a version



Always check: it will save you trouble

NEW Routine to *save/define* a version

Summary

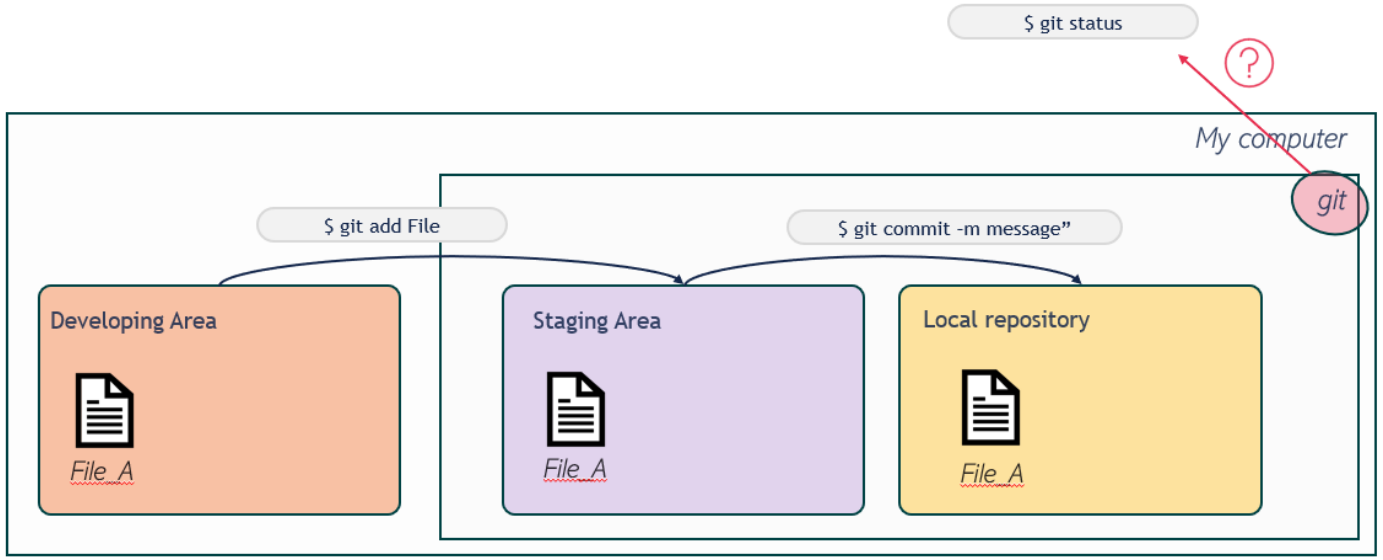


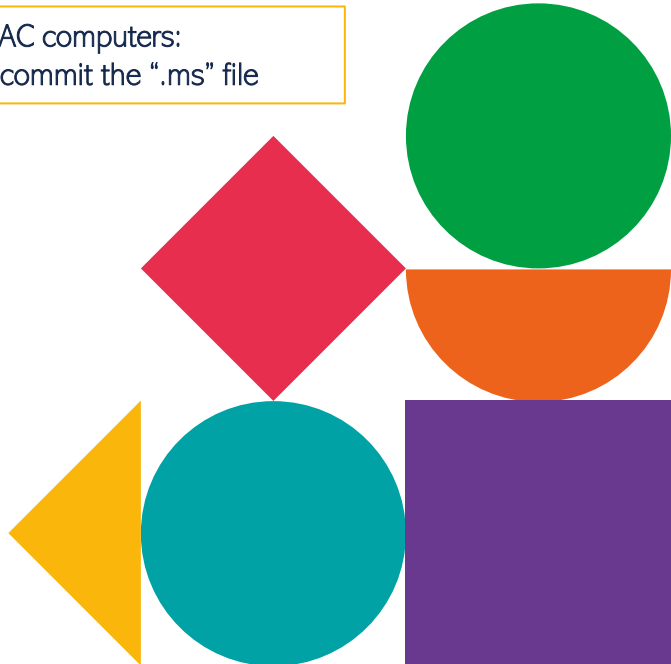
ACTIVITY

7. Update the commands-git.md

- Update the advised commands routine
- If necessary, add information about this in the dictionary.
- Check status at every step
- Check until you see "working tree clean" when asking for status

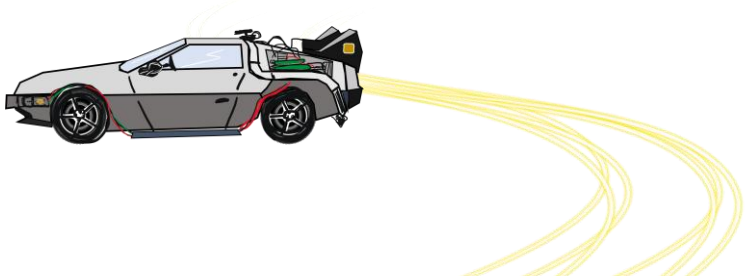
P.s. MAC computers:
don't commit the ".ms" file





Travel in time

How?

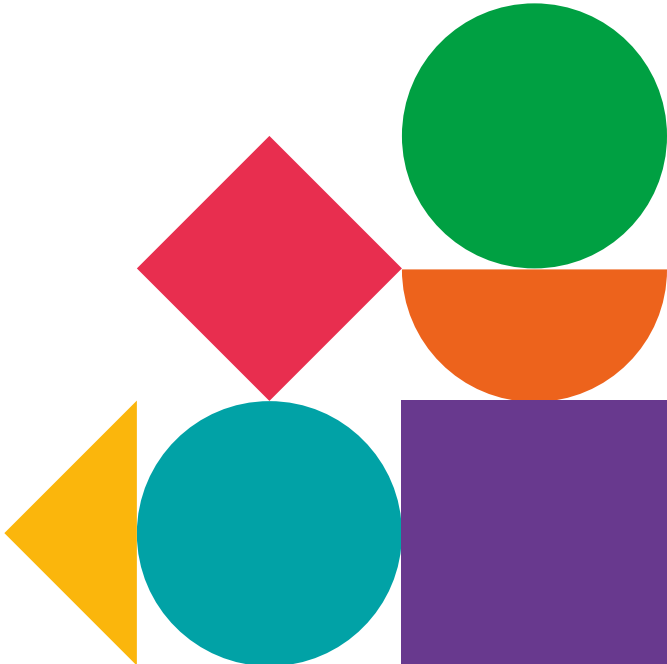


Activity

8. In pairs

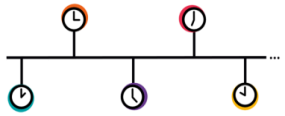
   - Each person has a different challenge
   - After solving your assignment, teach each other
   - Check how both challenges work together to allow you to see in the past

   - In the commands file annotate your new commands
   - In the dictionary annotate about the usability
   - Decide if you need one commit or two and do it
   - Don’t forget to check the status



Challenge 1

- How can I see the history of the timeline?



user$ git log

-- help

--oneline

-n <nº>

--name-only

--abbrev-commit

Your timeline

- Summary of all versions
- Chronological order

  top = most recent

  bottom = oldest

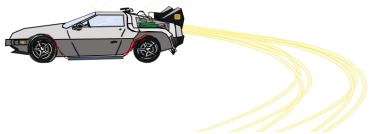
```
commit e2d7e9a0b4614a6bee6b3ffd7583237125671dc
Author: username <user@xyz.com>
Date: Wed Jan 01 01:23:45 2020 +0200

    The informative commit message
```



Challenge 2

- How can I see differences between current and past versions?



```
user$ git diff <old> <new>
```

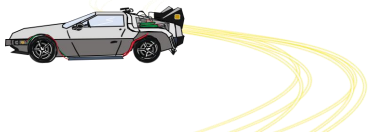
Prints a combined version

- + add lines
- - deleted or edited lines
- [ ] no changes

```
bpiereck@LaptopBruna:/mnt/c/Users/Bruna/OneDrive - VIB/Documents/Courses/Git_class/Class_may2024$ git diff 7de3da6 e1fde99
diff --git a/file_test.txt b/file_test.txt
index 8a9de4a..d781282 100644
--- a/file_test.txt
+++ b/file_test.txt
@@ -1,2 +1,3 @@
 My unmodifed liine
-Line to be deleted
+
+line added
```

Challenge 2

- How can I see differences between current and past versions?

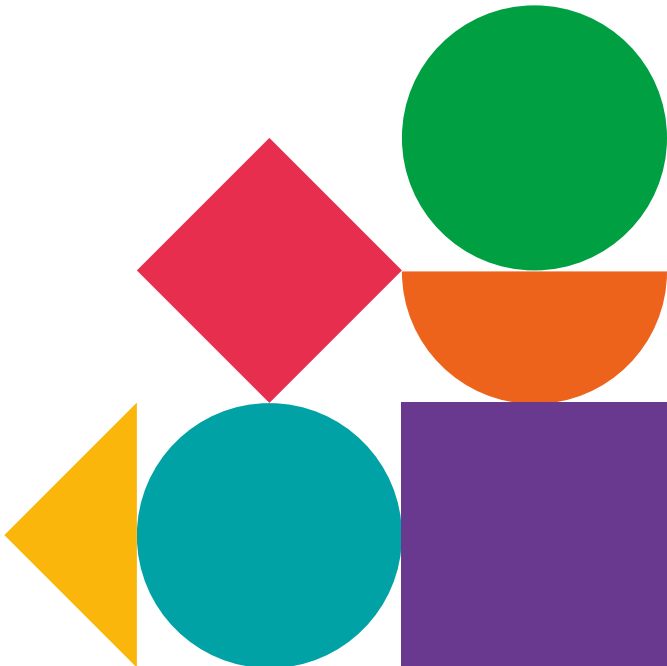


```
user$ git diff <old> <new>
```

```
user$ git show <old> <new>
```

Prints separate versions + info

- 1st older file
  - Commit
  - Text
- 2nd more recent file
  - Commit
  - Text



Challenge 2



Prints separate versions

- 1st older file
  - Commit
  - Text
- 2nd more recent file
  - Commit
  - Text

```
commit 7de3da6629e0a2e740ef224c755a5213e07d0e1c
Author: bruna.piereckmoura@vib.be <bruna.piereckmoua@vib.be>
Date:   Tue May 14 11:35:33 2024 +0200

    1st commit of 2nd file

diff --git a/file_test.txt b/file_test.txt
new file mode 100644
index 0000000..8a9de4a
--- /dev/null
+++ b/file_test.txt
@@ -0,0 +1,2 @@
+My unmodifed liine
+Line to be deleted

commit e1fde99c77cb101c243d7096a97c10ae5e000230 (HEAD -> master)
Author: bruna.piereckmoura@vib.be <bruna.piereckmoua@vib.be>
Date:   Tue May 14 11:36:07 2024 +0200

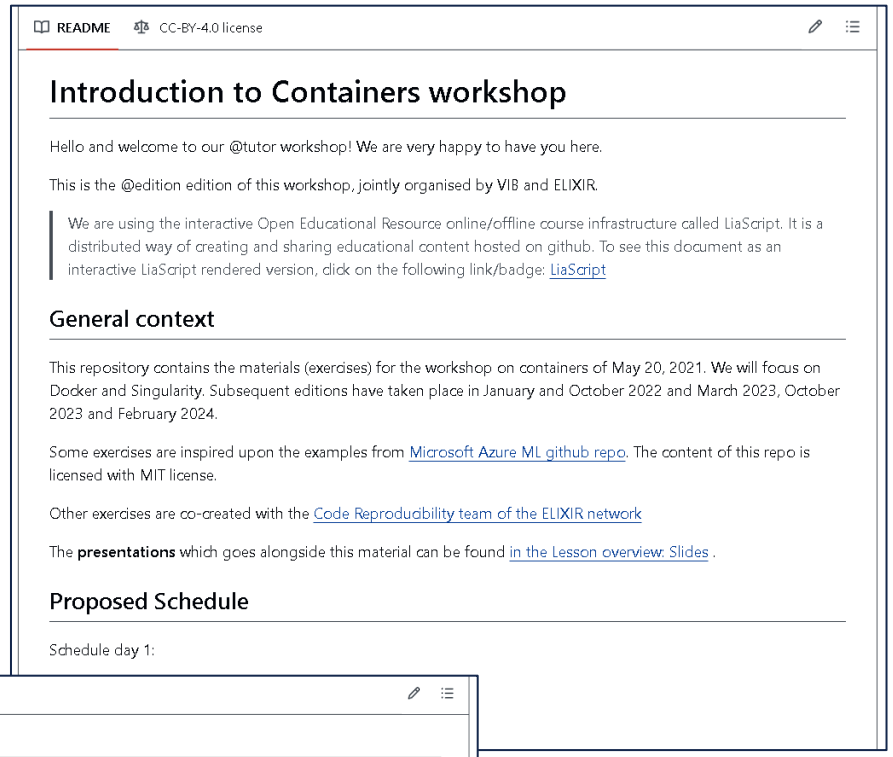
    2nd commit of 2nd file

diff --git a/file_test.txt b/file_test.txt
index 8a9de4a..d781282 100644
--- a/file_test.txt
+++ b/file_test.txt
@@ -1,2 +1,3 @@
 My unmodifed liine
-Line to be deleted
+
+line added
```

Special files

Important

- README.md file
  - Detailed description
    - Project
    - Usage
    - Installations



README CC-BY-4.0 license

Introduction to Containers workshop

Hello and welcome to our @tutor workshop! We are very happy to have you here.

This is the @edition edition of this workshop, jointly organised by VIB and ELIXIR.

> We are using the interactive Open Educational Resource online/offline course infrastructure called LiaScript. It is a distributed way of creating and sharing educational content hosted on github. To see this document as an interactive LiaScript rendered version, click on the following link/badge: LiaScript

General context

This repository contains the materials (exercises) for the workshop on containers of May 20, 2021. We will focus on Docker and Singularity. Subsequent editions have taken place in January and October 2022 and March 2023, October 2023 and February 2024.

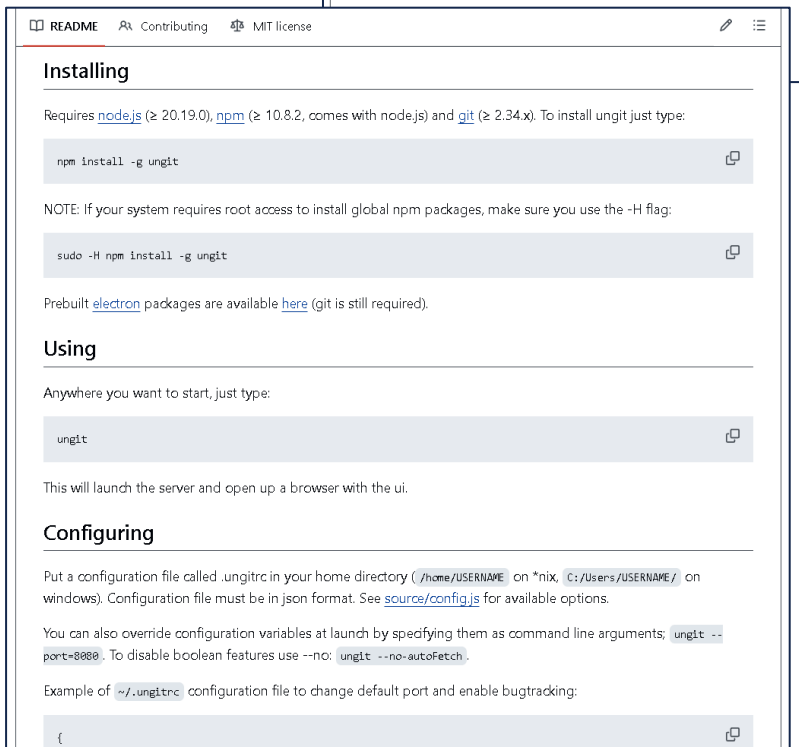
Some exercises are inspired upon the examples from Microsoft Azure ML github repo. The content of this repo is licensed with MIT license.

Other exercises are co-created with the Code Reproducibility team of the ELIXIR network

The **presentations** which goes alongside this material can be found in the Lesson overview: Slides .

Proposed Schedule

Schedule day 1:



README Contributing MIT license

Installing

Requires node.js (≥ 20.19.0), npm (≥ 10.8.2, comes with node.js) and git (≥ 2.34.x). To install ungit just type:

```
npm install -g ungit
```

NOTE: If your system requires root access to install global npm packages, make sure you use the -H flag:

```
sudo -H npm install -g ungit
```

Prebuilt electron packages are available here (git is still required).

Using

Anywhere you want to start, just type:

```
ungit
```

This will launch the server and open up a browser with the ui.

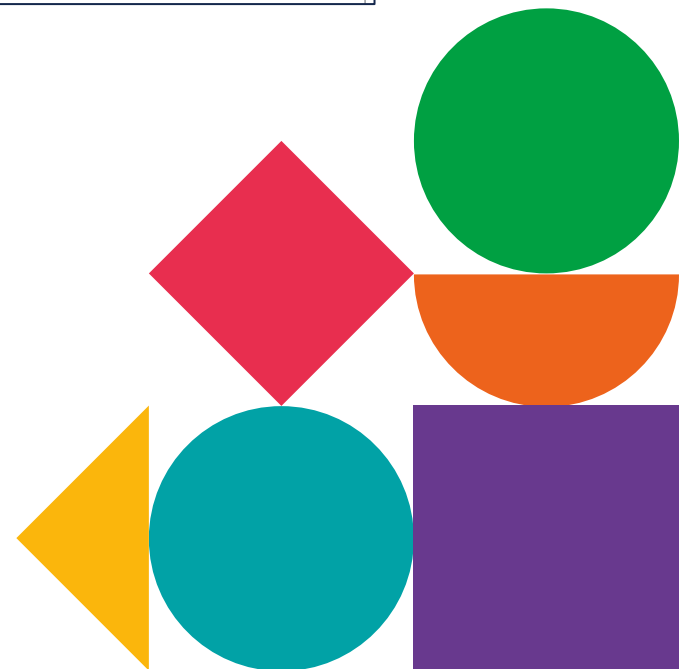
Configuring

Put a configuration file called .ungitrc in your home directory (`/home/USERNAME` on *nix, `C:/Users/USERNAME/` on windows). Configuration file must be in json format. See source/config.js for available options.

You can also override configuration variables at launch by specifying them as command line arguments; `ungit --port=8080`. To disable boolean features use --no: `ungit --no-autoFetch`.

Example of `~/.ungitrc` configuration file to change default port and enable bugtracking:

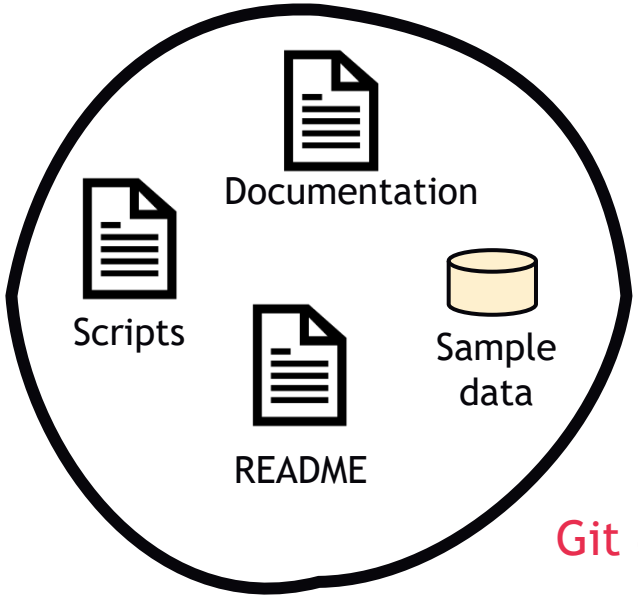
```
{
```



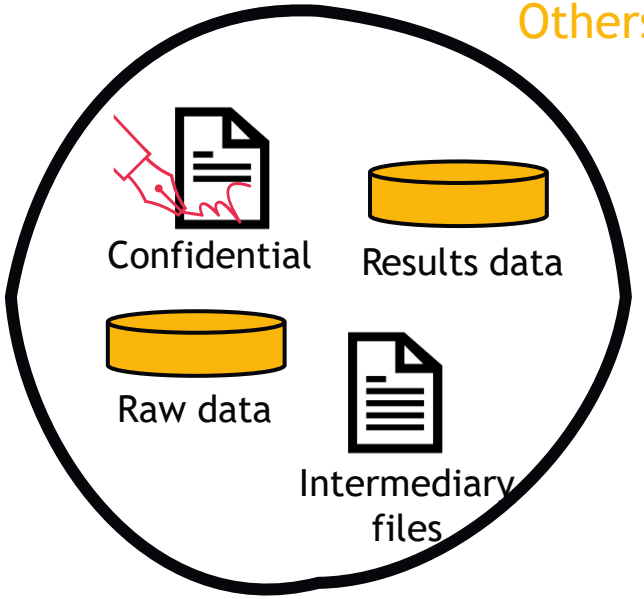
Special files

Important

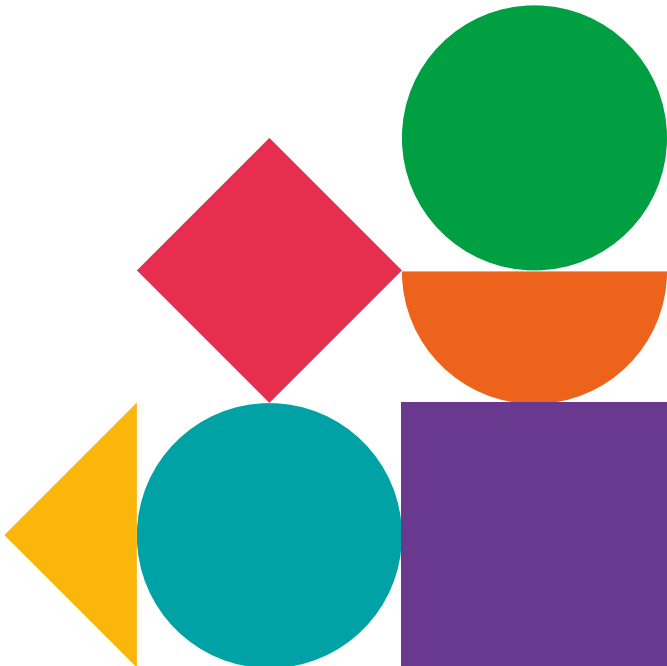
- README.md file
  - Detailed description
    - Project
    - Usage
    - Installations
- .gitignore file
  - List of files to be ignored
    - Data
    - Backup
    - Intermediate files
    - Confidential files



Git & GitHub



Others



Special files

Important

- README.md file
  - Detailed description
    - Project
    - Usage
    - Installations
- .gitignore file
  - List of files to be ignored
    - Data
    - Backup
    - Intermediate files
    - Confidential files

- Ignore
  - Files
  - Folders
- Using REGEX
  - *.csv = all CSV files
  - !datasample.csv = except this file
  - # text = make comments



ACTIVITY

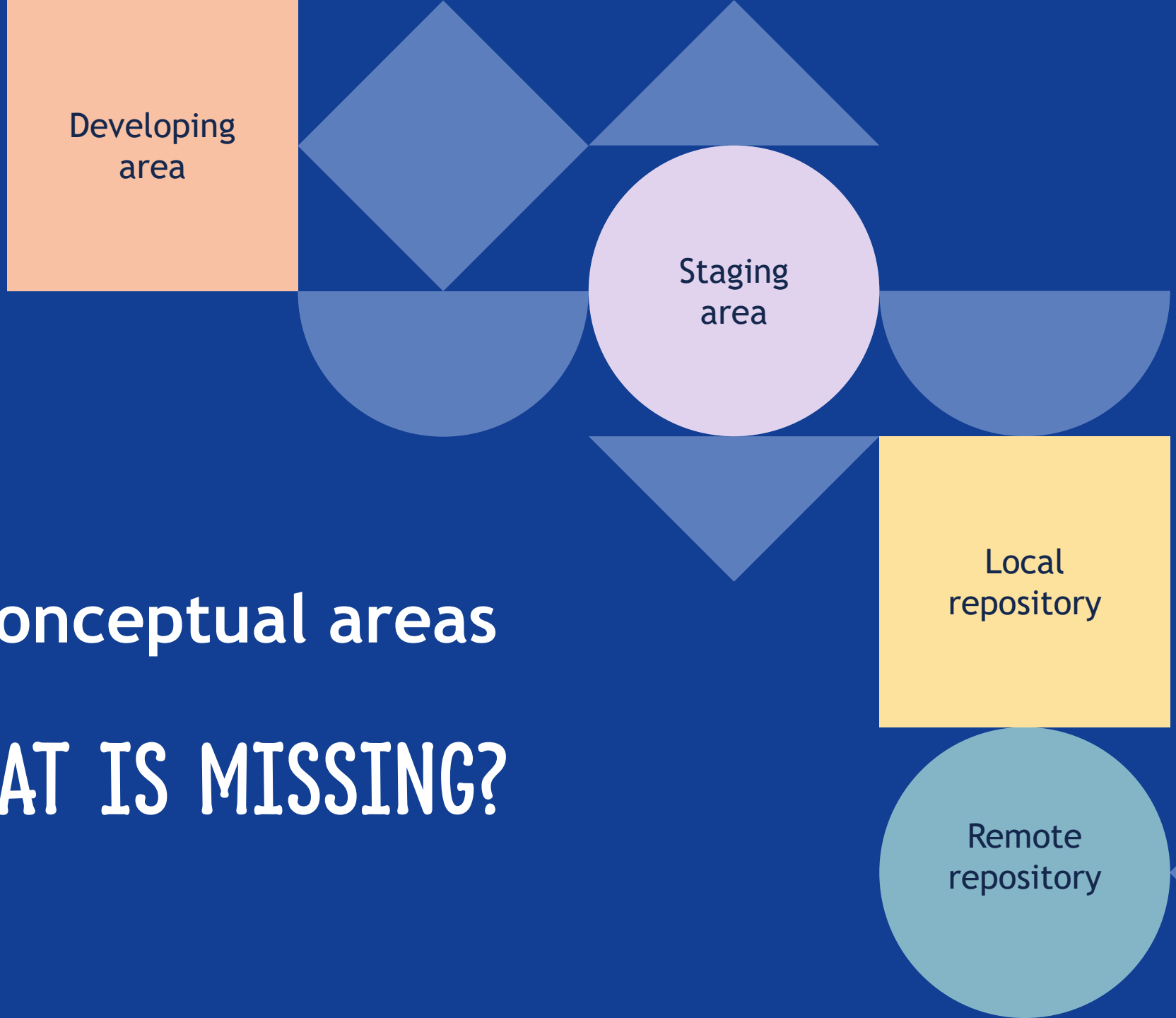
9. Create 2 new files README.md and .gitignore

- Describe your project in the README.md
- Add the explanation of each file/folder you have in your repository (dictionary, commands, etc.)

- Create a .gitignore (name exactly as it is here)
- Add a new image (check the status, should be untracked)
- Include the image name in the gitignore , check the status, should not be mentioned)
- For MAC computers, ignore the *.ms file

- Commit the changes, think if you will commit all together or separately.





4 conceptual areas

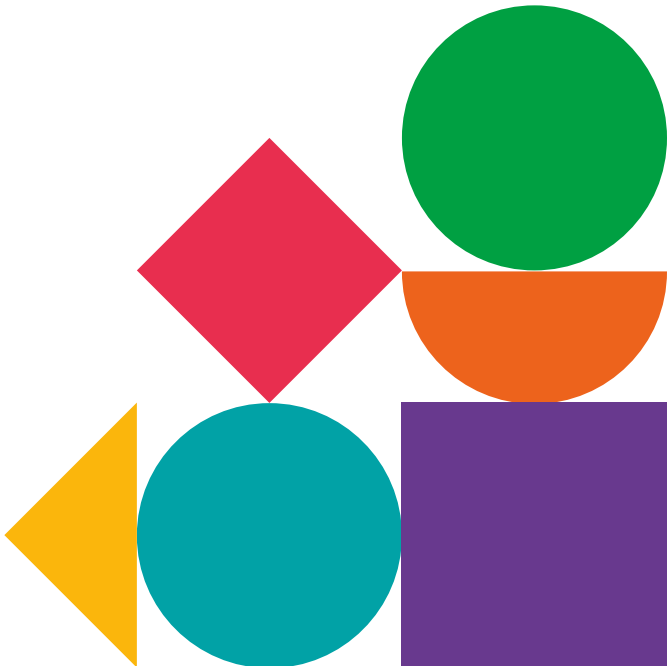
WHAT IS MISSING?

Local to Remote

Connecting with GitHub

Allowing communication local >>> remote

- SSH key
  - An authenticator
  - Connects PC and Remote location
    - Gives access to your computer
  - Strong “memorizable” password
  - Requested at each connection



Allowing communication local >>> remote

- SSH key
  - One per computer
  - An authenticator
  - Connects PC and Remote location
    - Gives access to your computer
  - Strong “memorizable” password
  - Requested at each connection

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

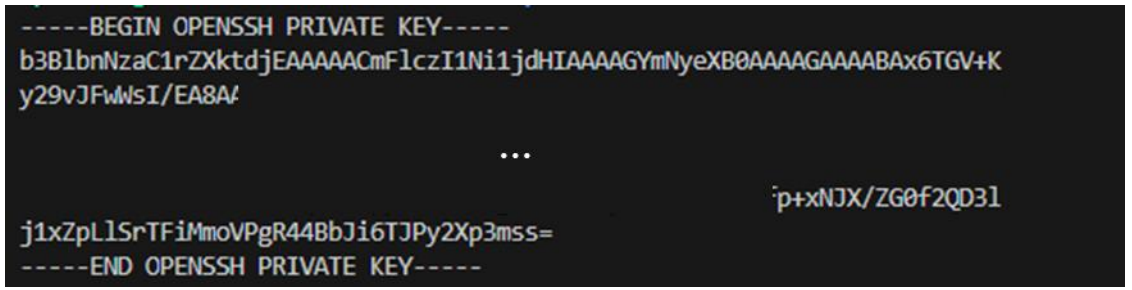
Checks:
- Is the key in your folder (it shouldn‘t)
- Is the key in the `~/.ssh folder` (the write place)

```
bpiereck@LAP0361:/mnt/c/Users/bruna.piereckmoura/OneDrive - VIB/Docume
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/bpiereck/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
The key's randomart image is:
+--[ED25519 256]--+
|   ..  . o ...   |
|    ... + * . . E|
|     .=. % = + ..|
|     oooO @ + o o|
|     . =S* = + + |
|      o . . o *  |
|         . . . + |
|          . . o.+|
|             ..+o|
+----[SHA256]-----+
```

Allowing communication local >>> remote

- SSH key
  - One per computer
  - An authenticator
  - Connects PC and Remote location
    - Gives access to your computer
  - Strong “memorizable” password
  - Requested at each connection

private

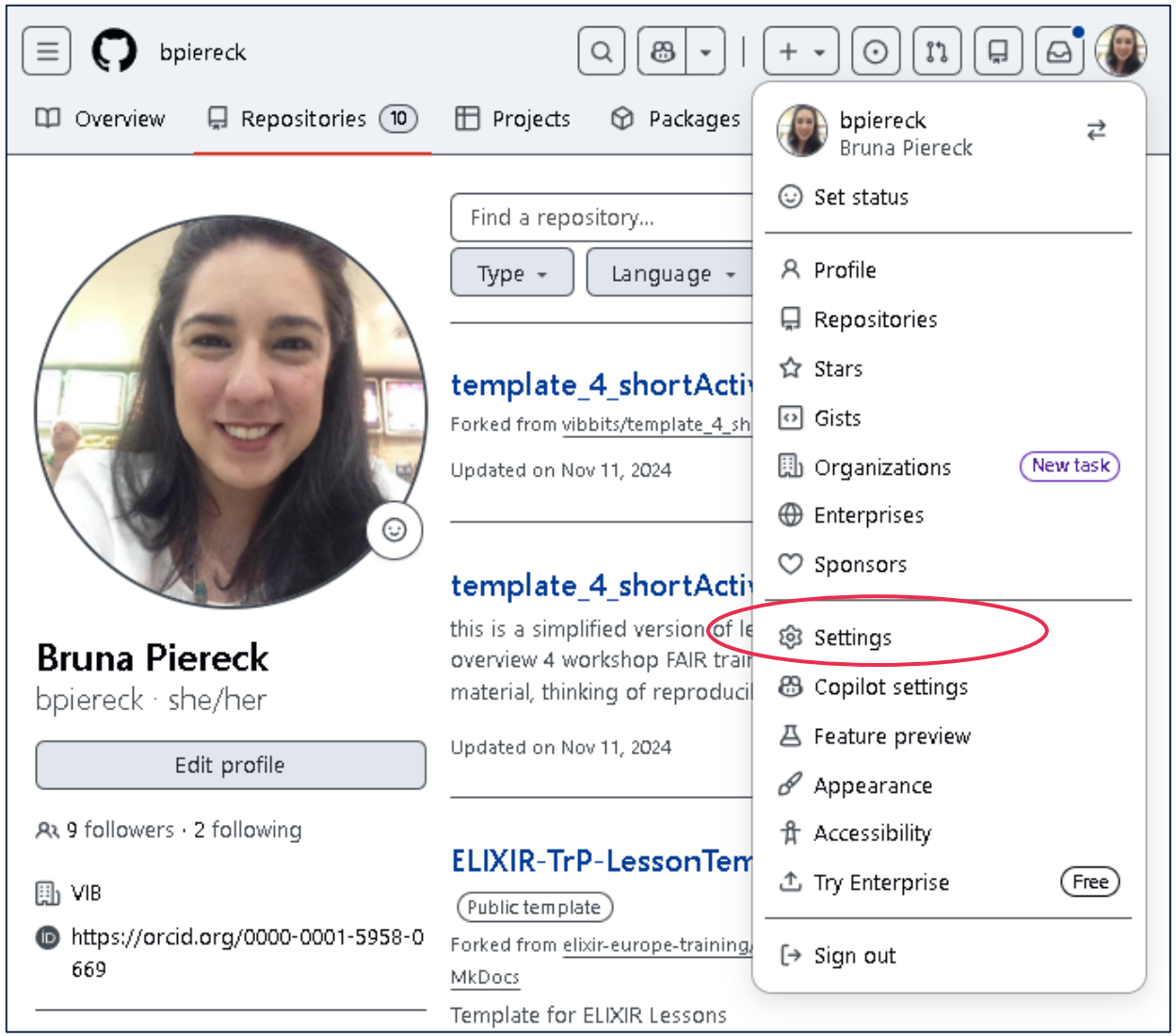


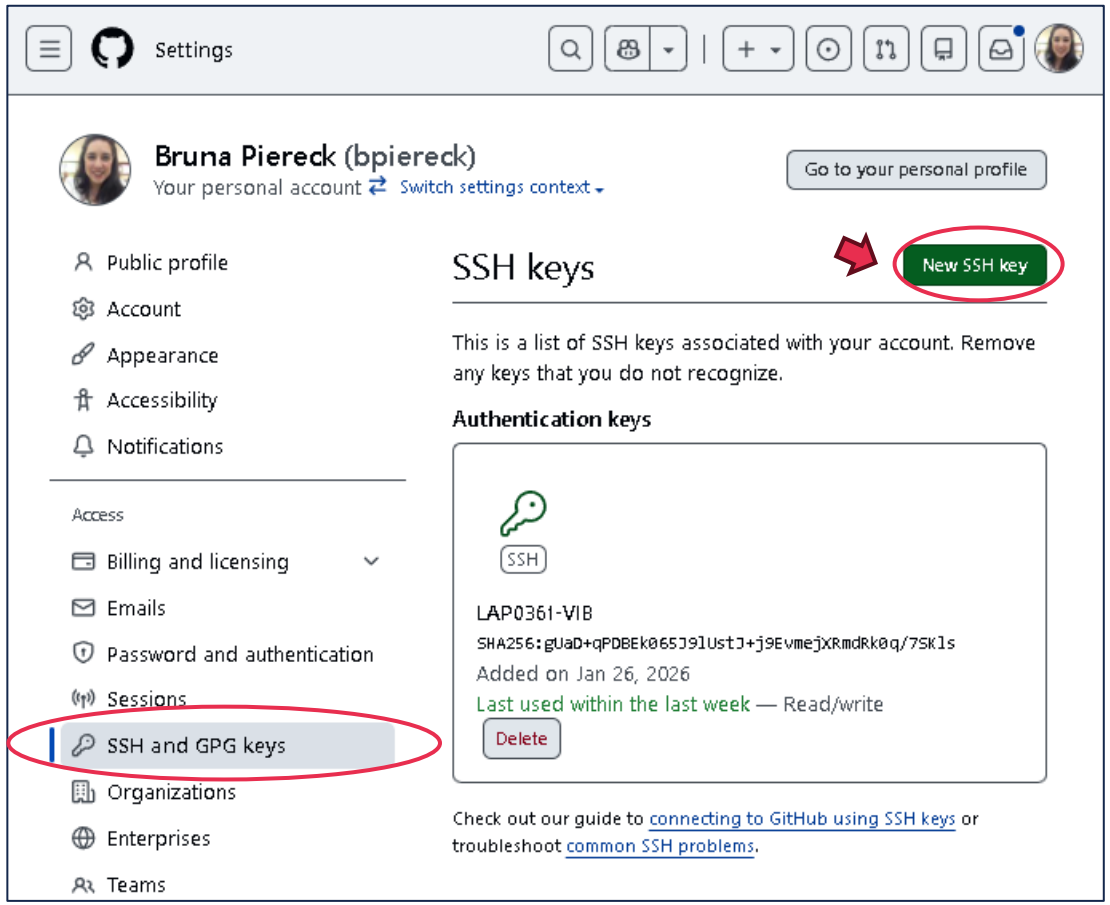
public

cat ~/.ssh/id_ed25519.pub

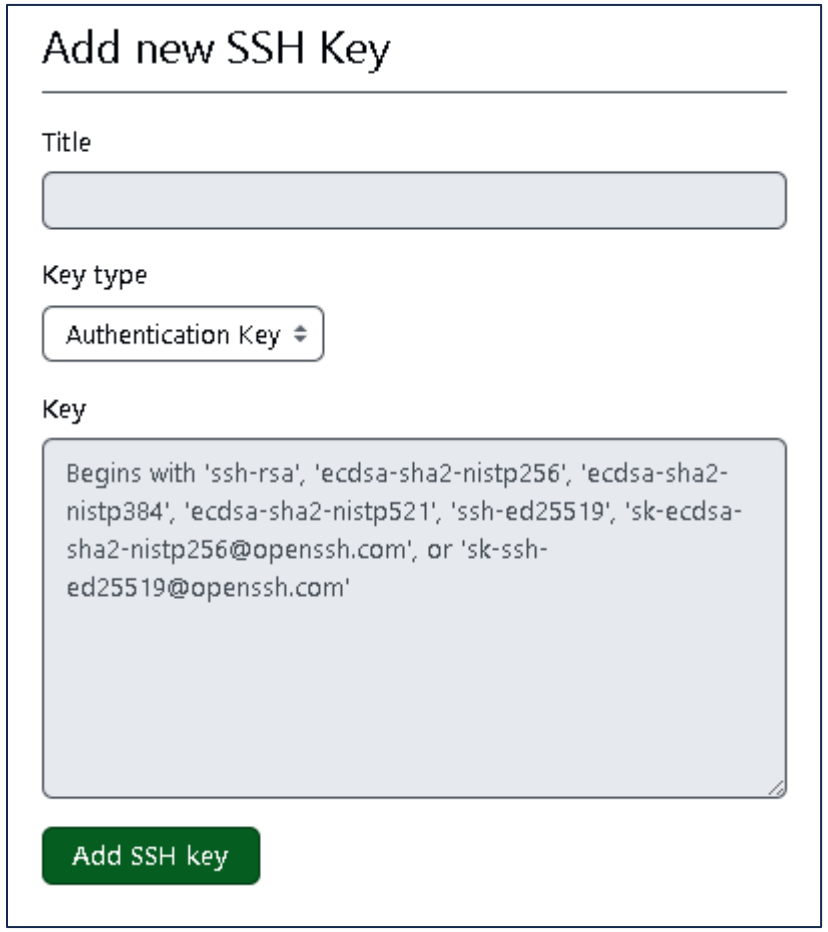
ssh-ed25519 AAAAC3Nza ……. ……. ……. ……. ……. ……. ……. ……. ……. TXQ bruna.piereckmorua@vib.be

Allowing communication local >>> remote





Allowing communication local >>> remote

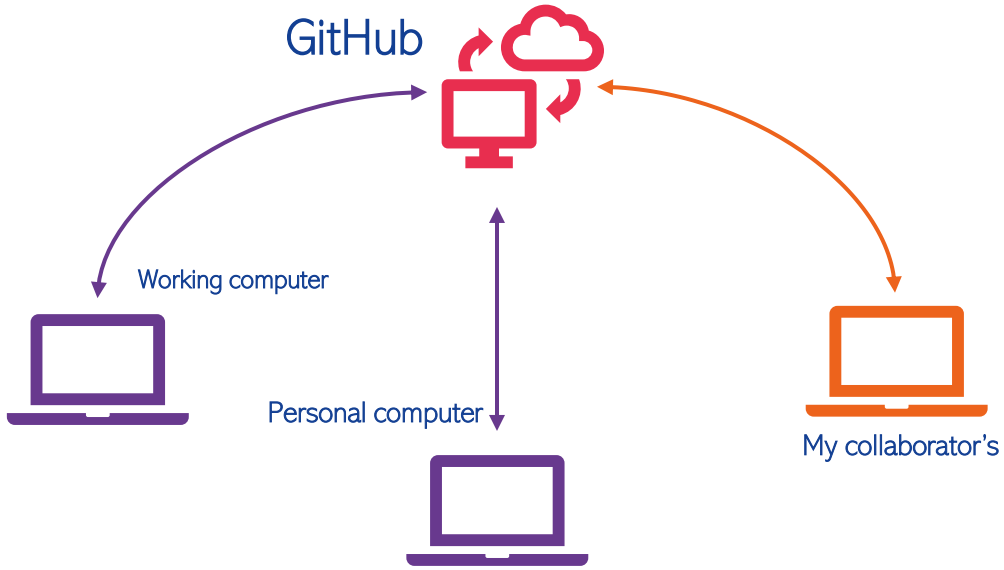


Names you can use

- Work computer
- VIB computer
- Personal computer
- LAP0361-VIB-computer

Names you should not use

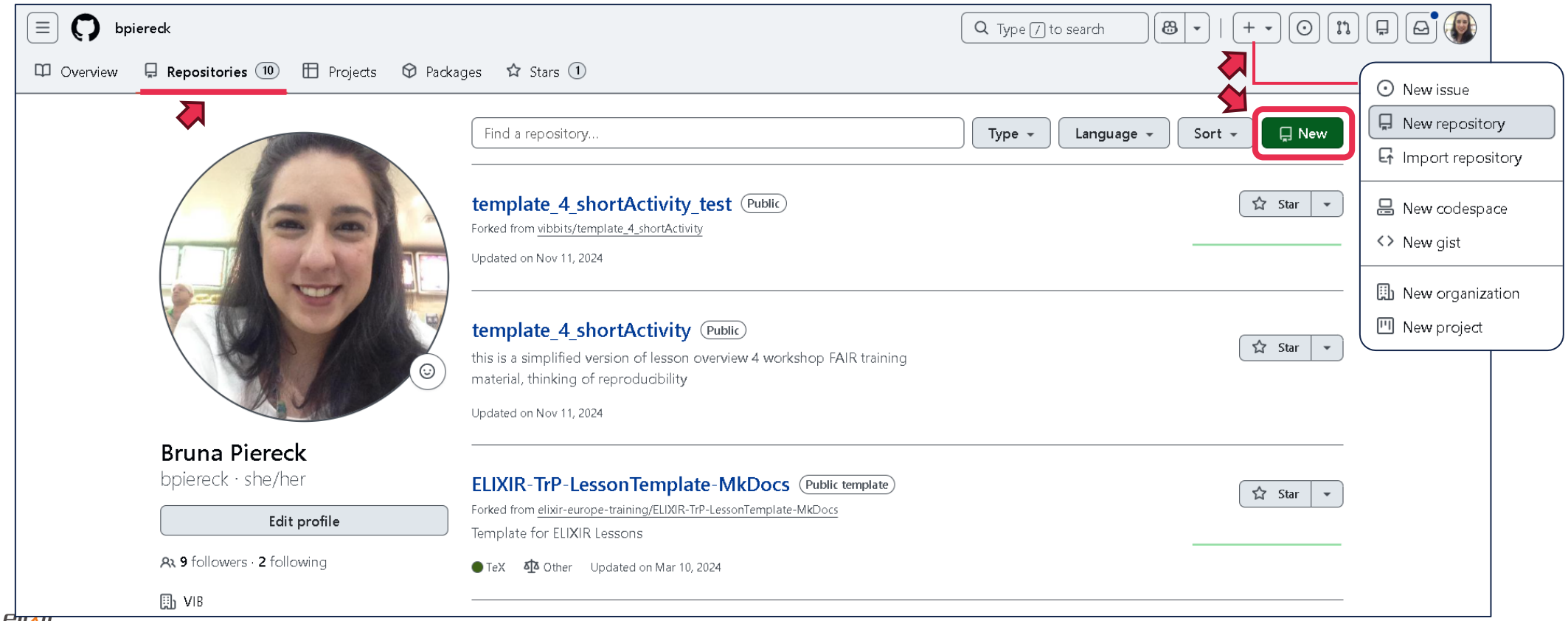
- GitHub course
- Training key
- Bruna key

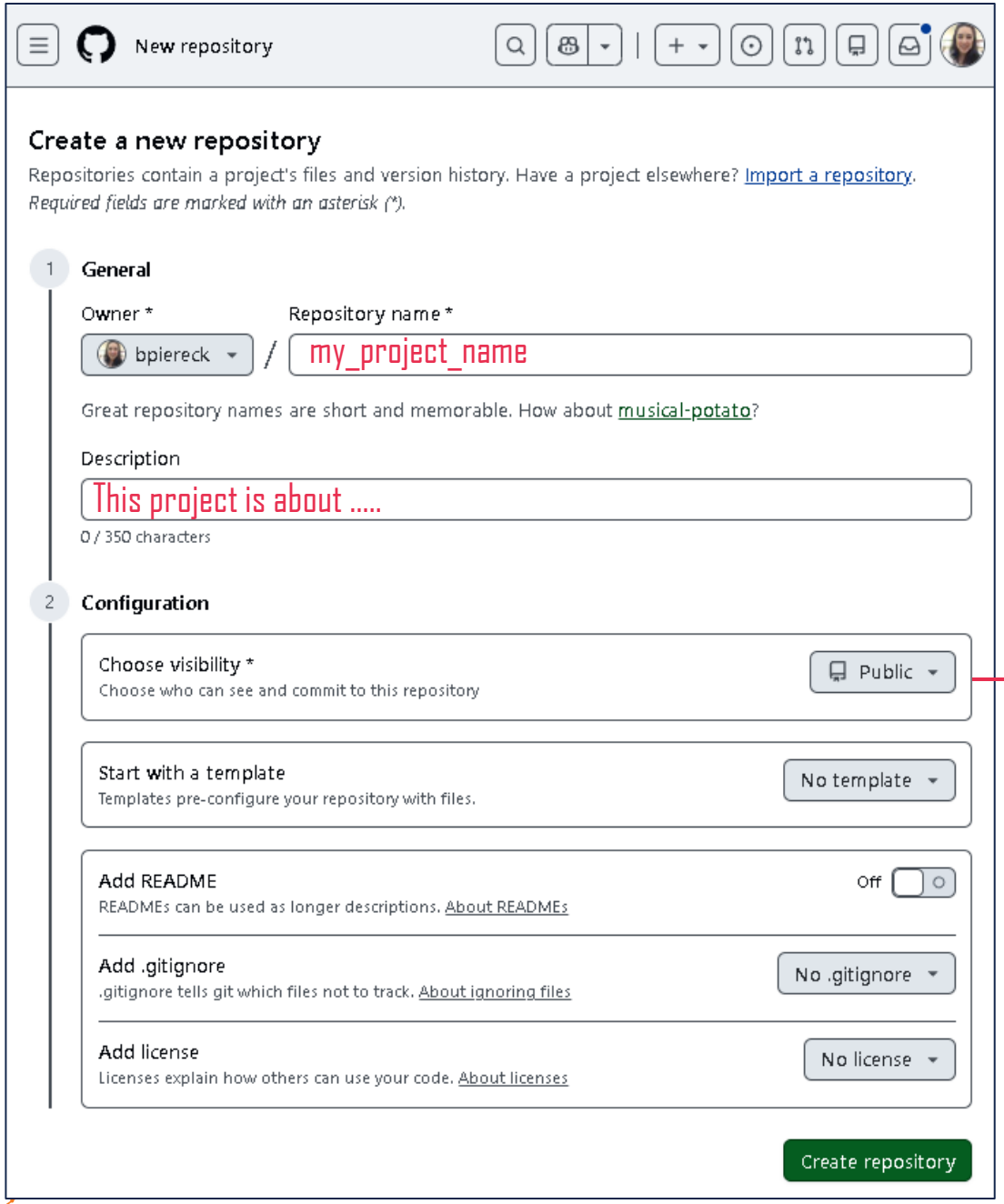


Create a remote repository

Situation 1: Backup a local repo in a remote repo

- Dedicated space for backup of your timeline
- One for each git project

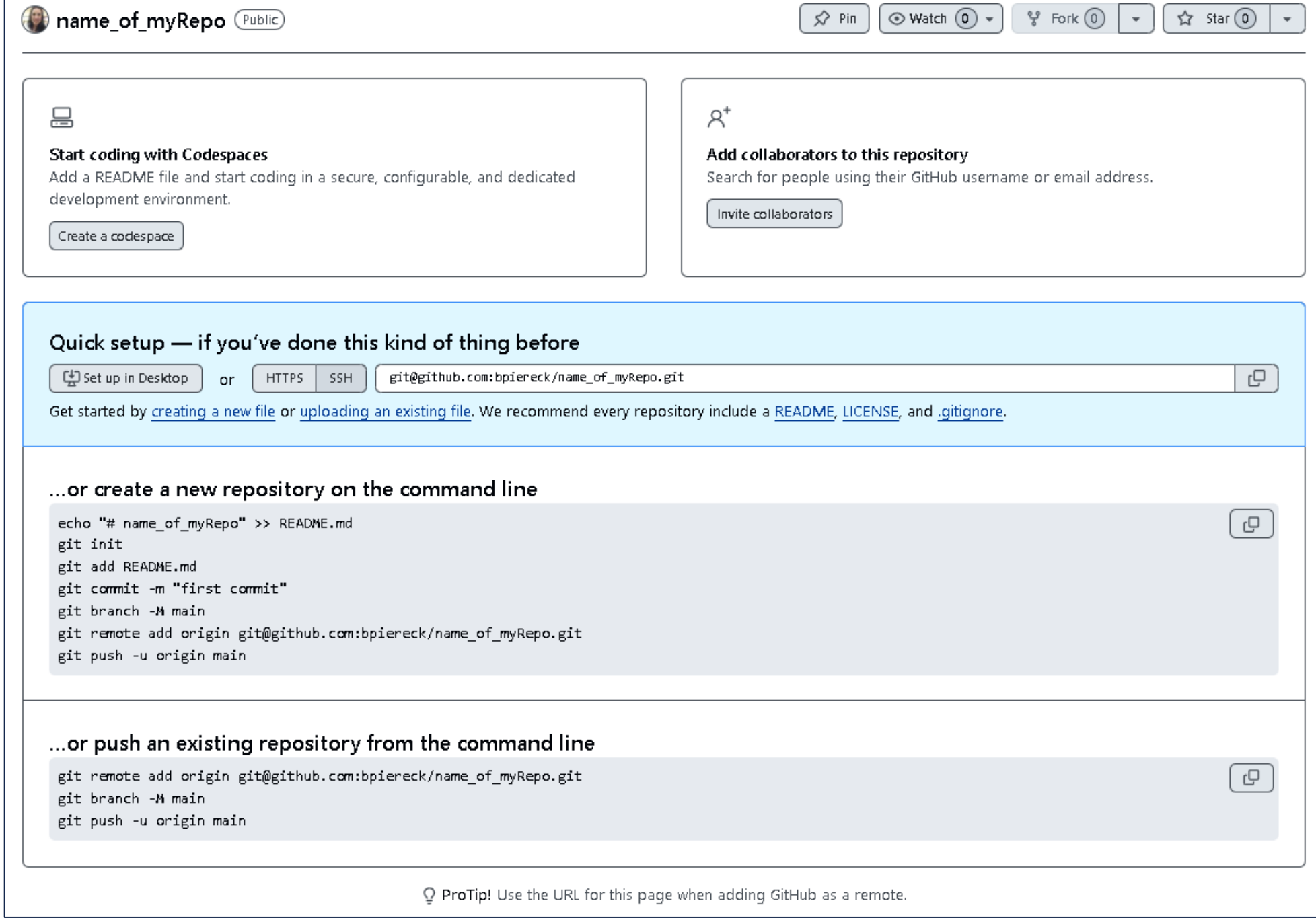






Don’t add ANY files in this step !

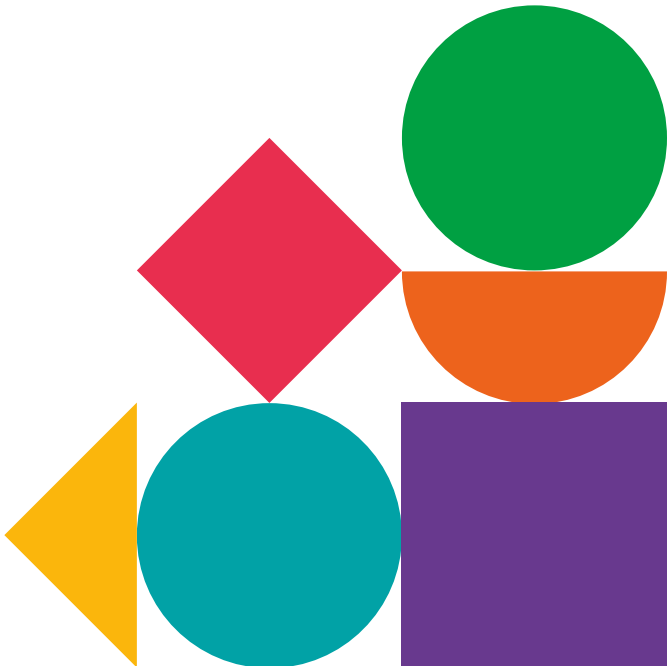
You need an empty repo.


name_of_myRepo
Public
Pin
Watch 0
Fork 0
Star 0
Start coding with Codespaces
Add a README file and start coding in a secure, configurable, and dedicated development environment.
Create a codespace
Add collaborators to this repository
Search for people using their GitHub username or email address.
Invite collaborators
Quick setup — if you've done this kind of thing before
Set up in Desktop
or
HTTPS
SSH
git@github.com:bpiereck/name_of_myRepo.git
Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.
...or create a new repository on the command line
echo "# name_of_myRepo" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:bpiereck/name_of_myRepo.git
git push -u origin main
...or push an existing repository from the command line
git remote add origin git@github.com:bpiereck/name_of_myRepo.git
git branch -M main
git push -u origin main
ProTip! Use the URL for this page when adding GitHub as a remote.

ACTIVITY

10. Annotate in your files

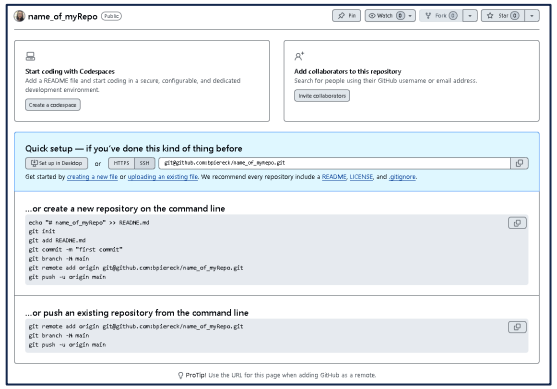
- About the ssh key
  - How to
  - Importance
- About how to create an empty repository
  - How to
  - When to do it
  - When can you create a non-empty repository



We have ...



Remote repository



Local repository



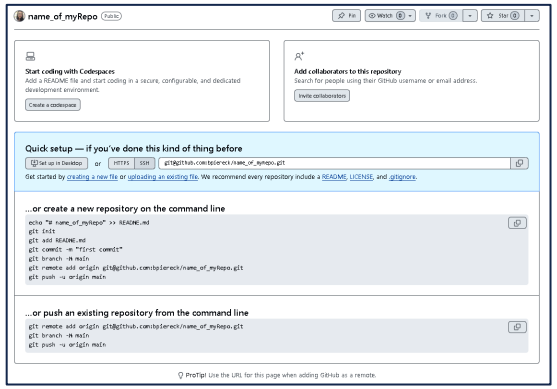


We need ...





Remote repository





Local repository



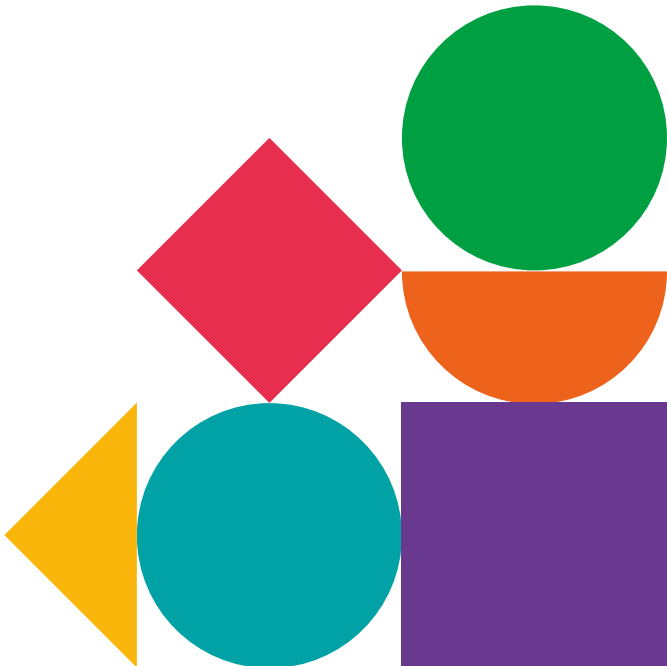


Connecting repositories



```
user$ git remote add origin <ssh address>
```

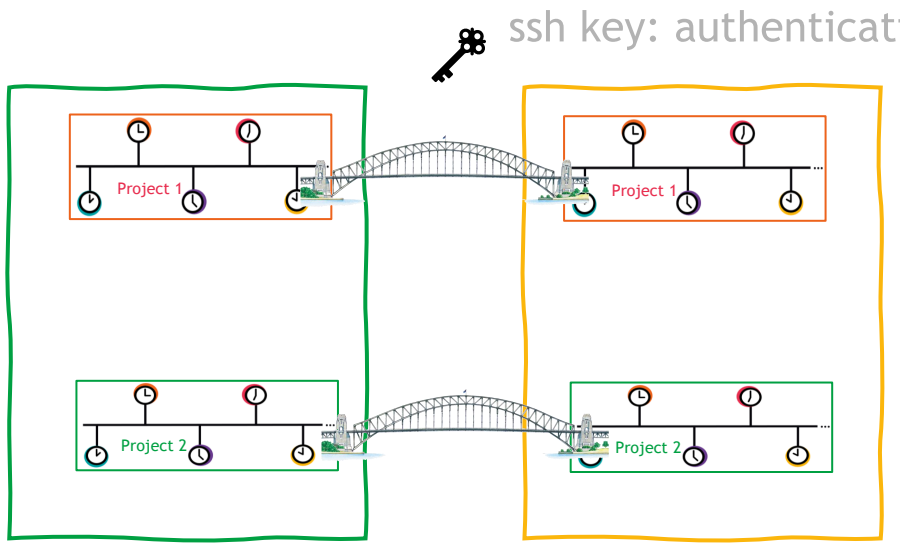
- Creates a “bridge” between two points
- Data is NOT automatically shared
- Back up ONLY for committed changes
- Just need to do it once



Connecting repositories

```
user$ git remote add origin <ssh address>
```

- Different repos = independent connections
- Same key



<ssh address> : location_project1

<ssh address> : location_project2

ACTIVITY

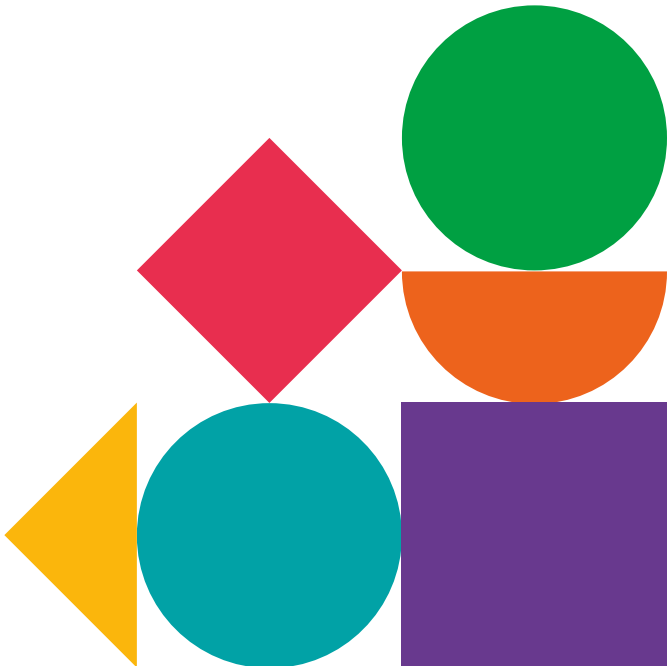
11. Connect your local repository to your remote repository

```
user$ git remote add origin <ssh address>
```

!!! Important

Follow the instructions git will print in the screen

- Annotate in your files
  - How to create a “bridge”
  - When to create it

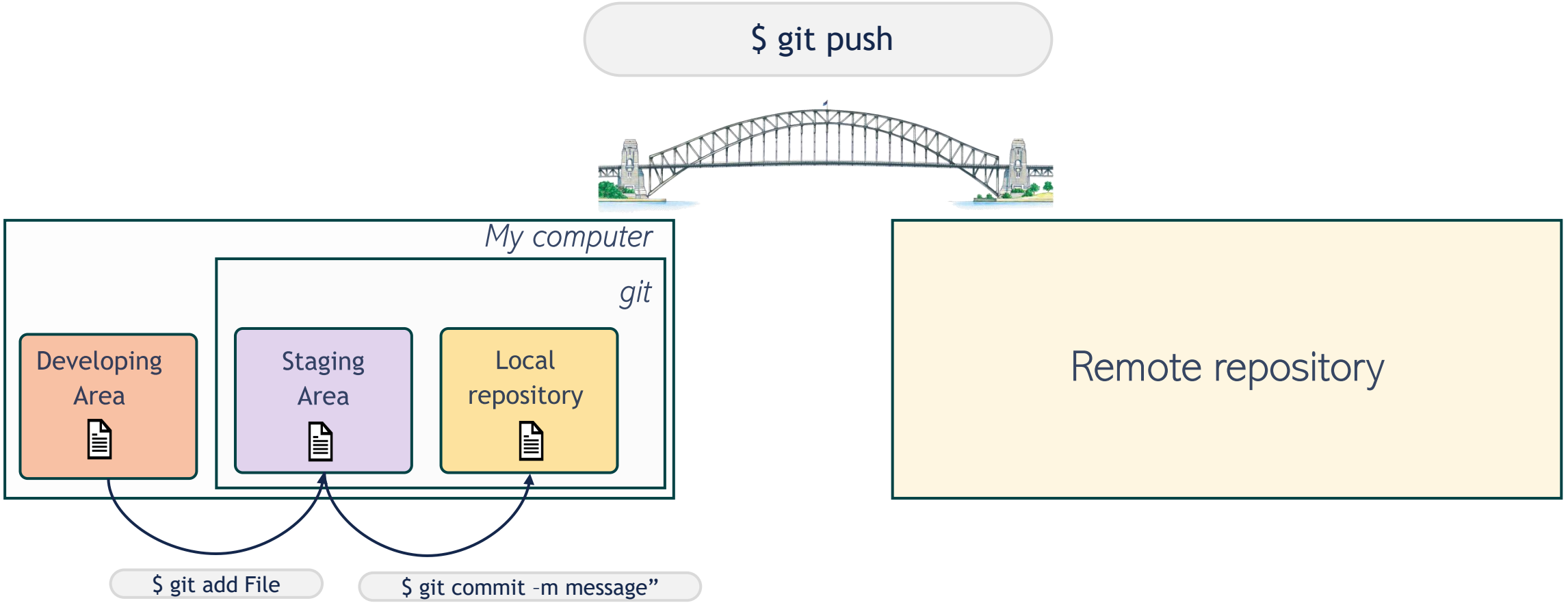


HOW DO WE TRANFER DATA TO REMOTE REPOSITORY?



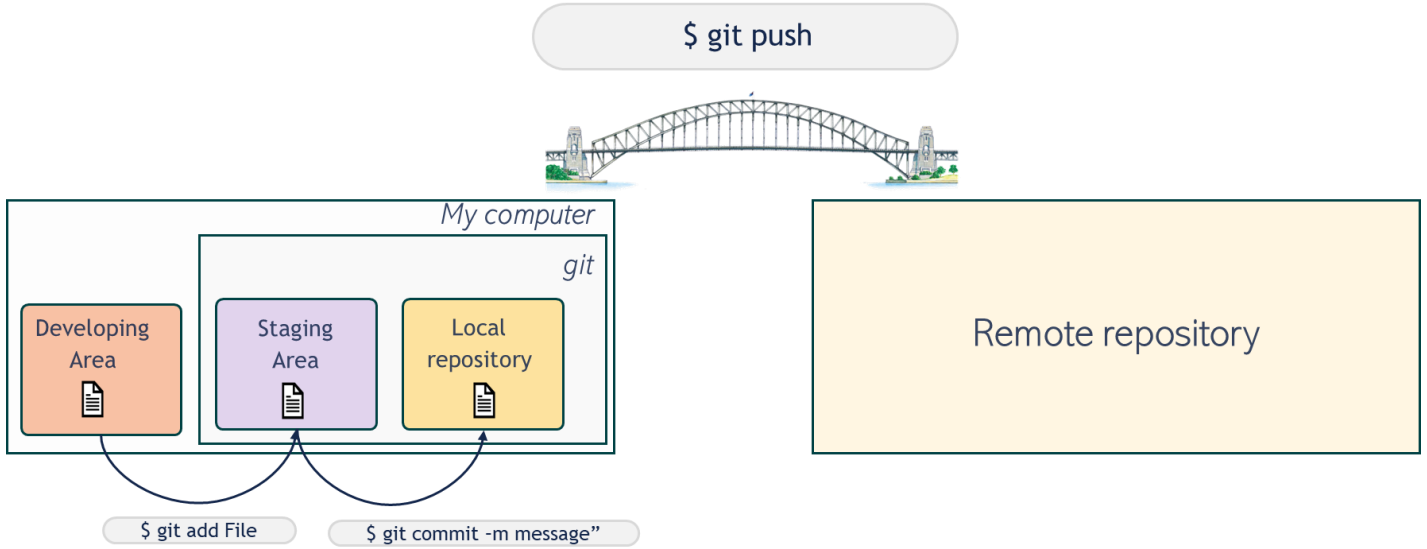
VIB elixir BELGIUM

Sync changes from local to remote

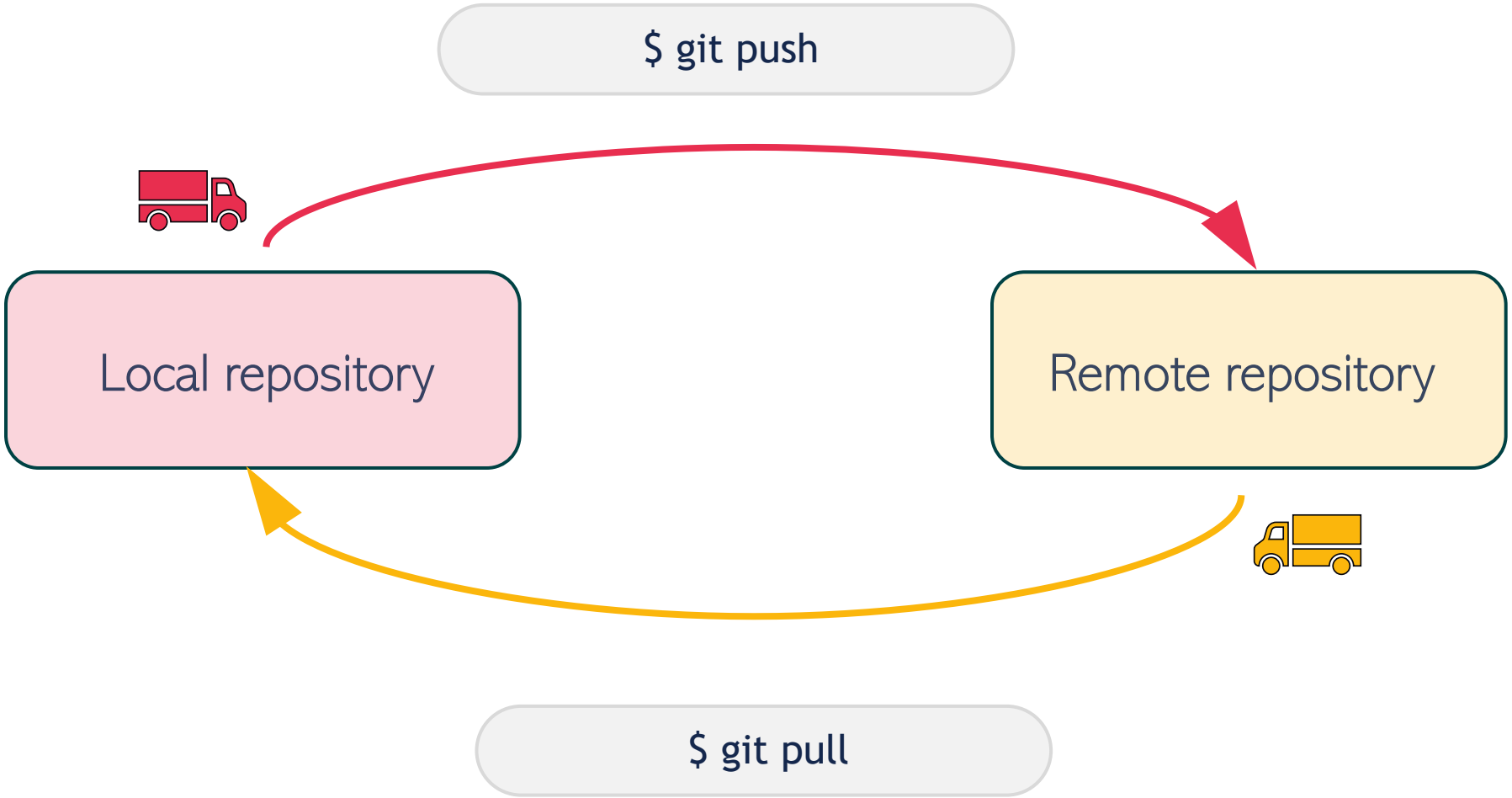


ACTIVITY

12. Push all your content to GitHub



A “bridge” works two ways



QUIZ

- When are PUSH and PULL useful?

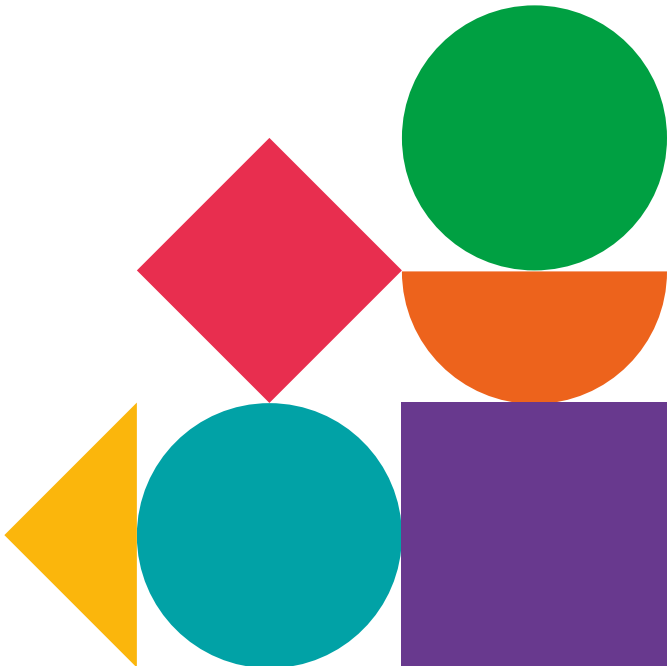
1. We push
   A. To create a back up of the versions in GitHub
   B. To share the changes with collaborators
2. We pull
   A. To see locally the changes made by collaborators
   B. To see locally changes that were done online (GitHub)

ACTIVITY

13. Check your files (part 1)
    - Check the status
    - Make sure your work tree is clean

```
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
```



ACTIVITY

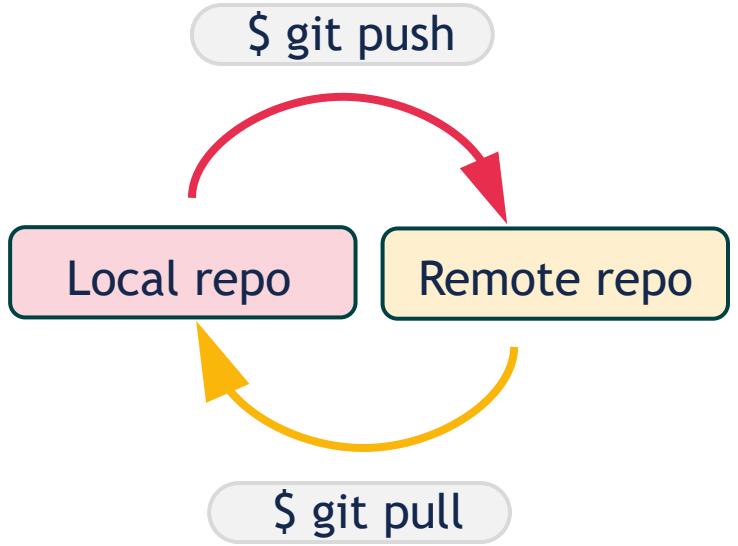
13. In GitHub annotate new concepts and commands (part 2)



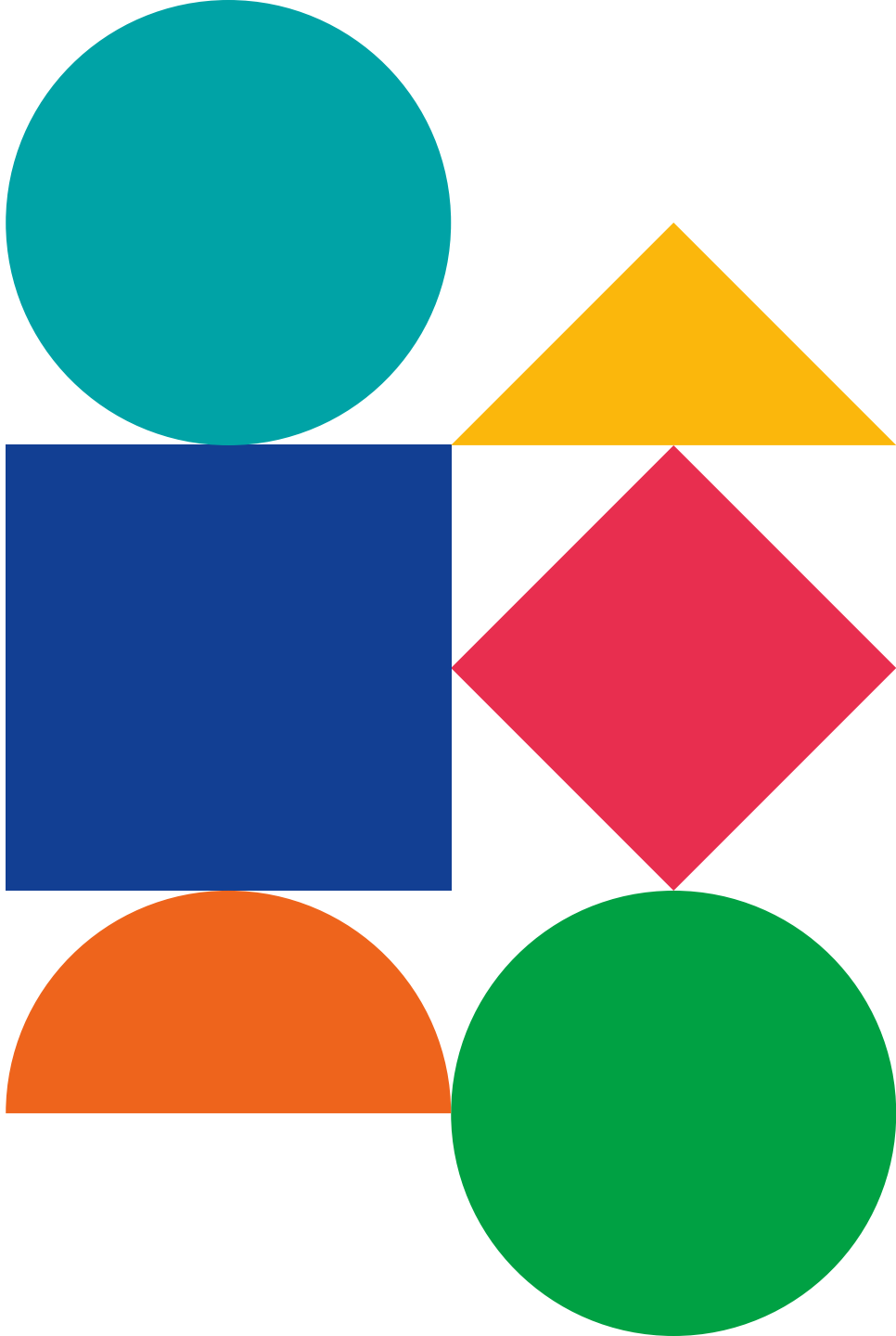
user$ git remote add origin <ssh address>

- How to send files to GitHub
  - How to create a “bridge”
  - When to create it
  - What are the commands to sync local & remote

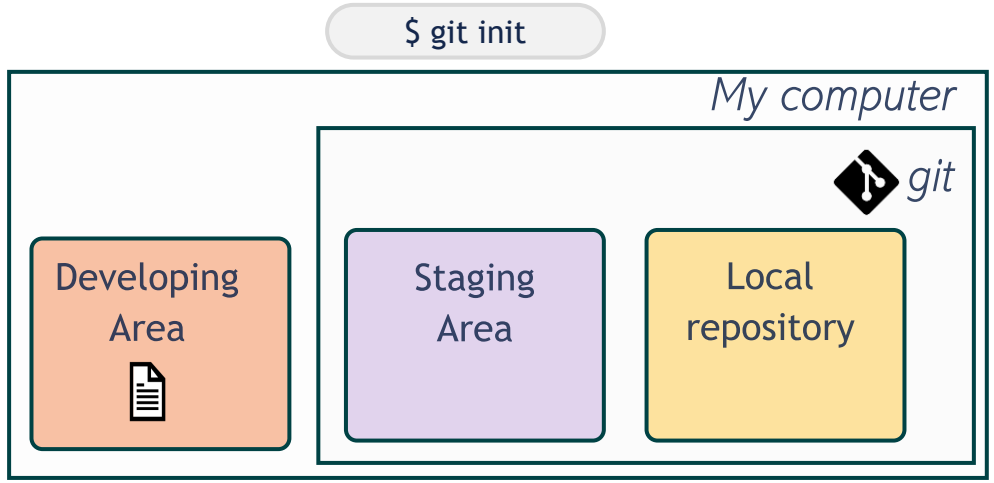
- Check the status
  - Sync your local repository



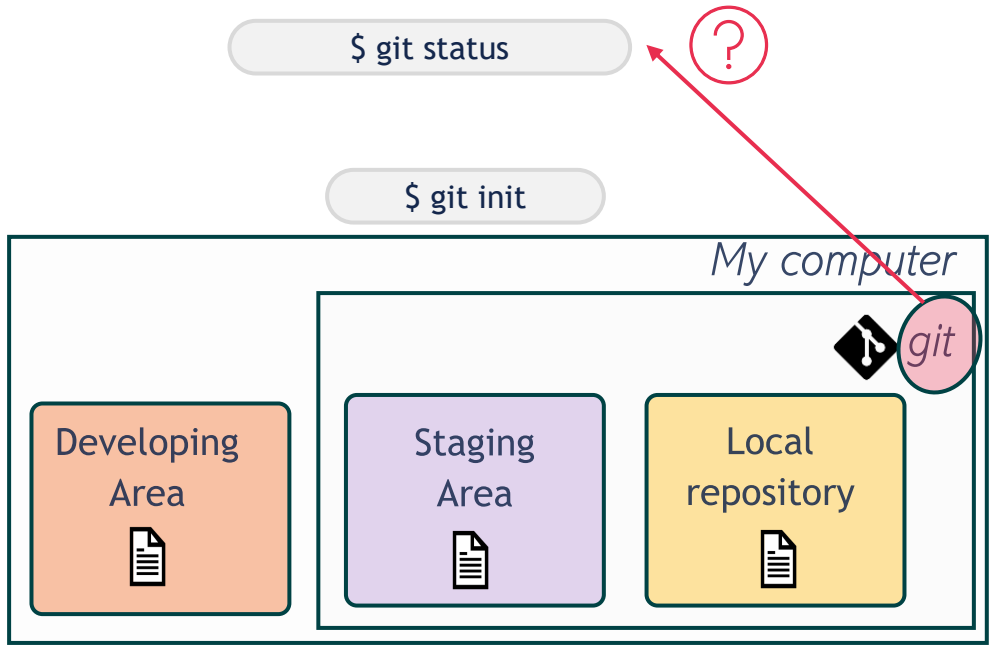
REVIEW



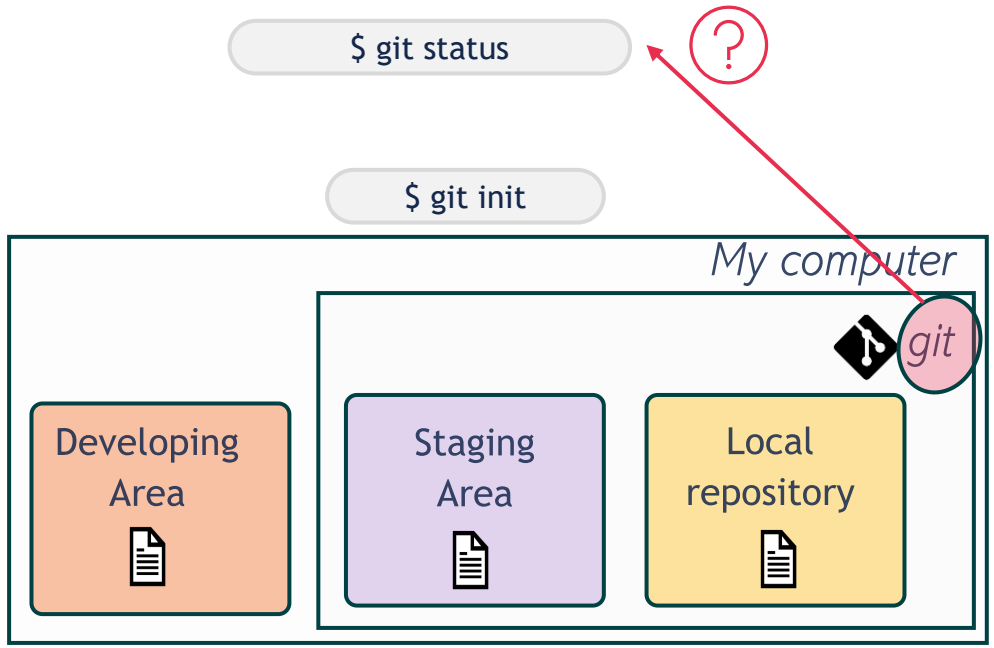
How I create a git repository? An empty timeline?

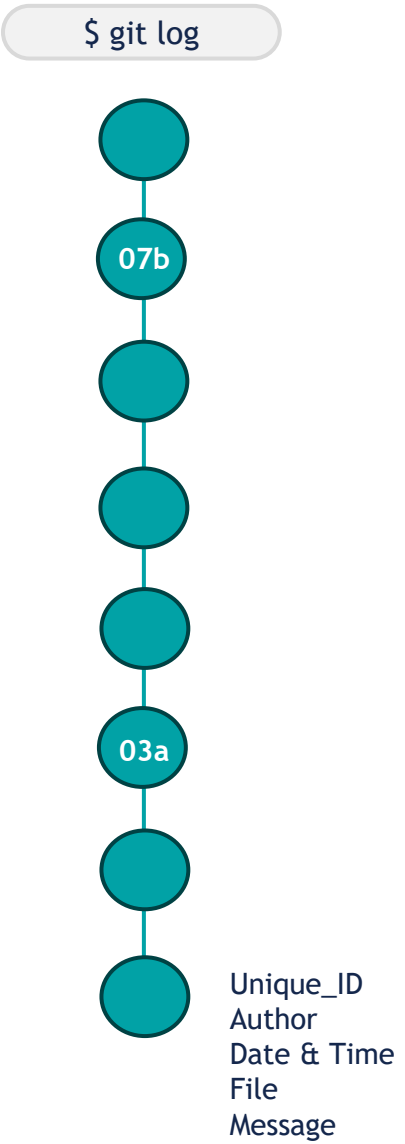


If I want to know in which conceptual area is the file?

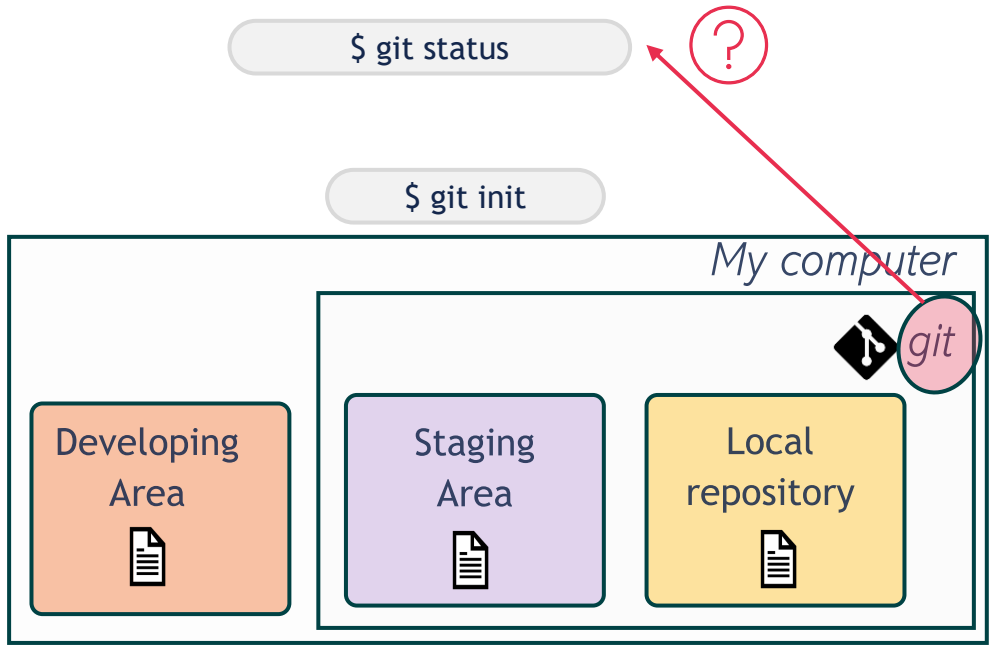


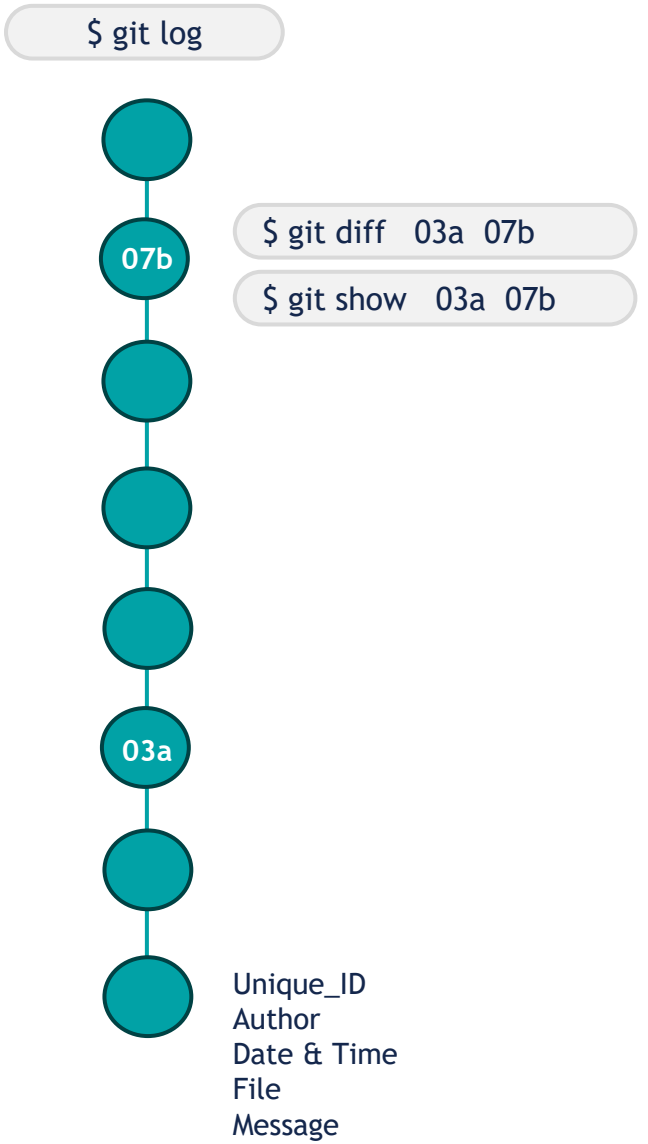
If I want to see all commits that I've done ?



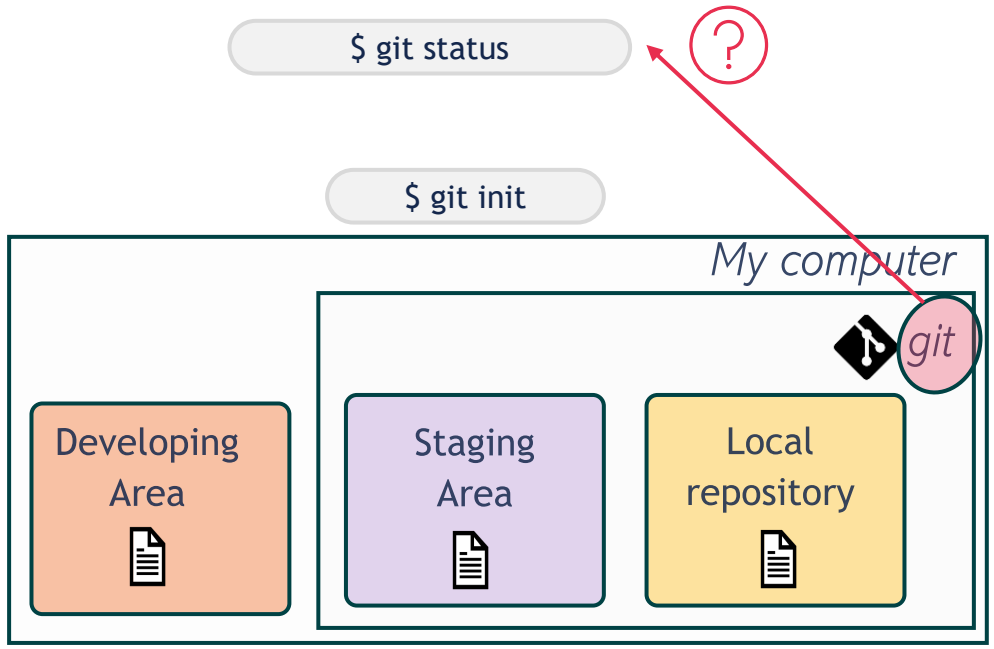


If I want to compare commits?

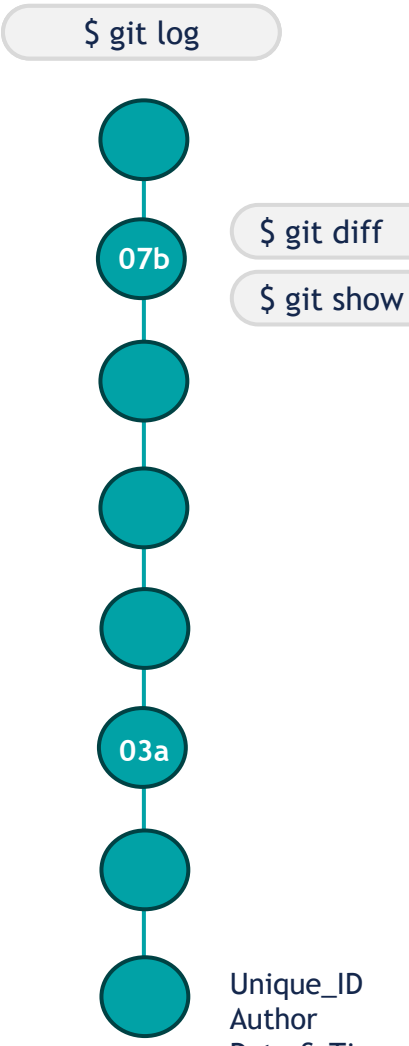




If I want to authorize connecting with GitHub, I need ?



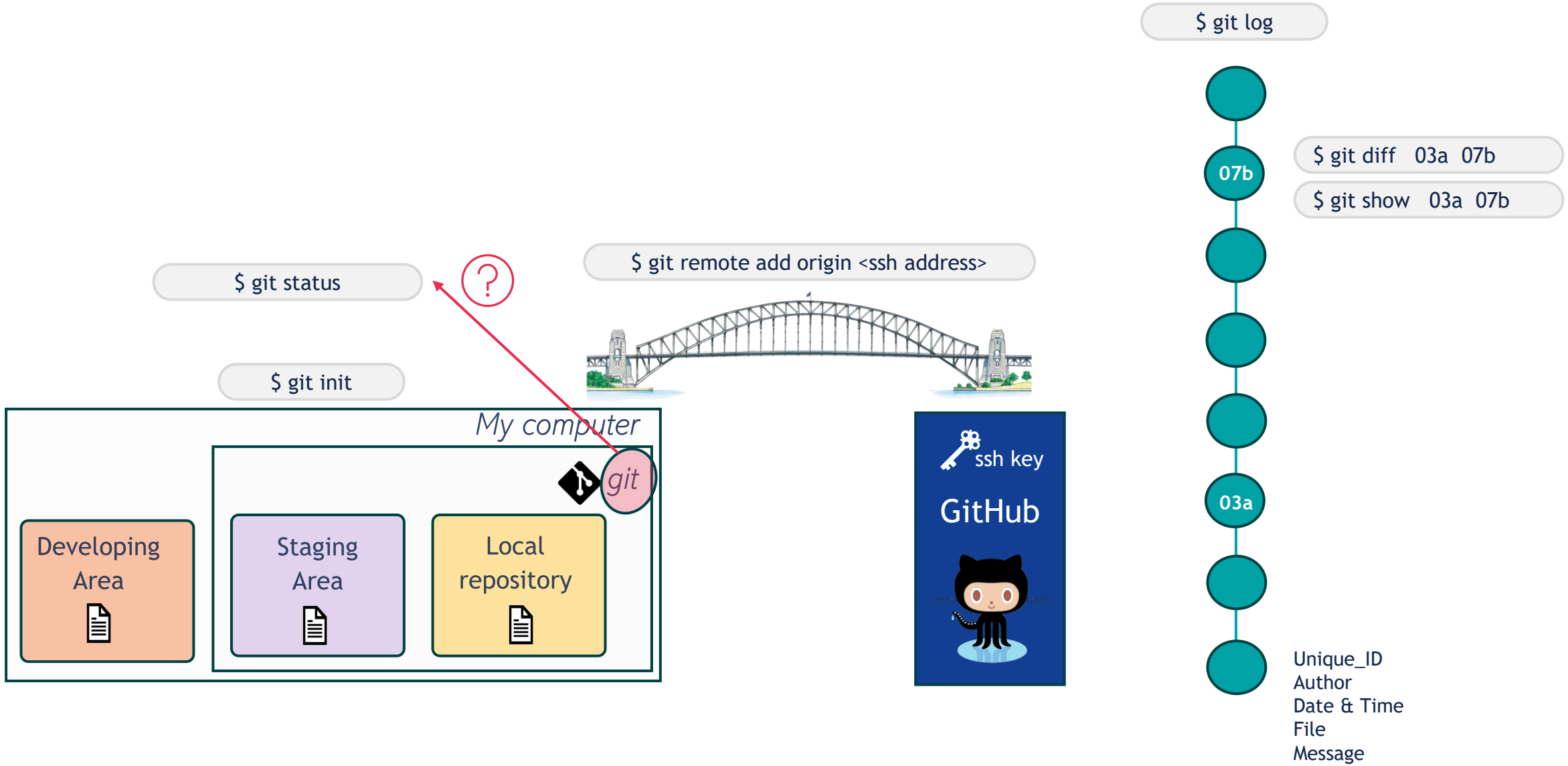




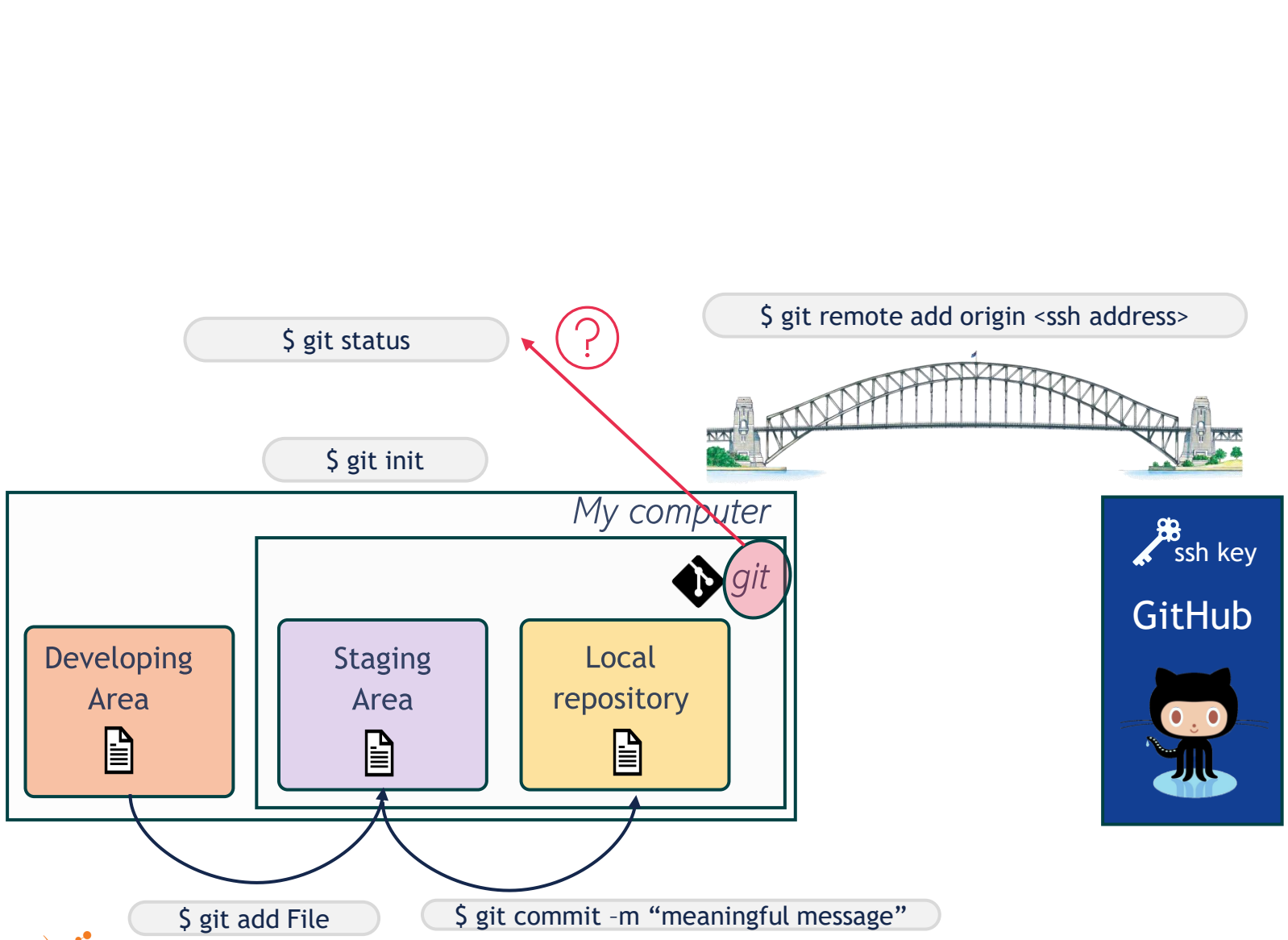
$ git diff 03a 07b

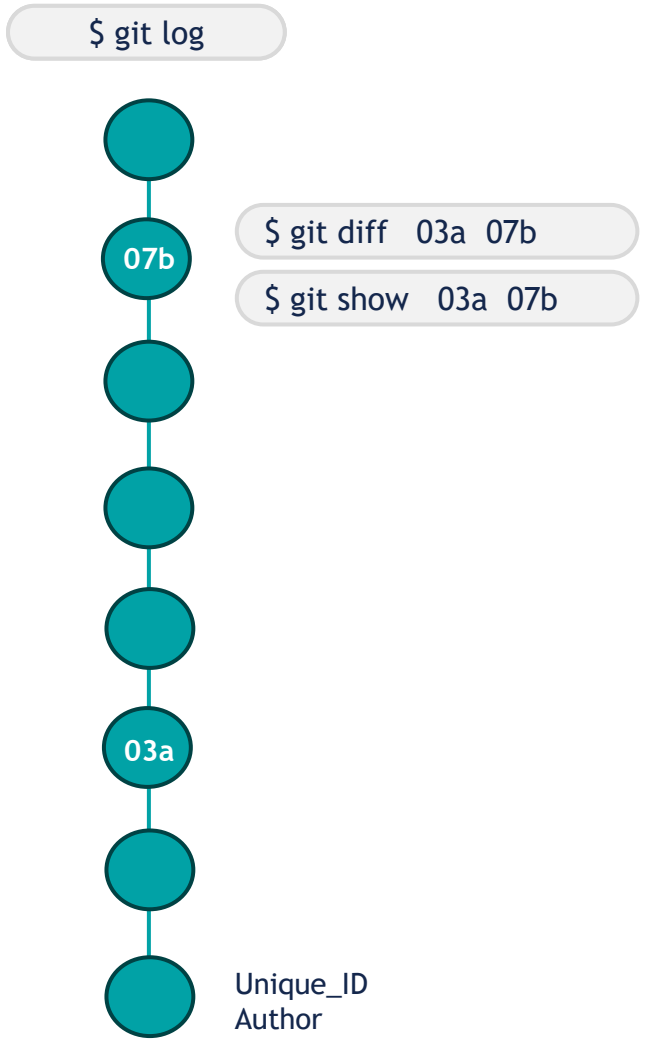
$ git show 03a 07b

Before transferring data to GitHub I need to create a channel. How do I do?

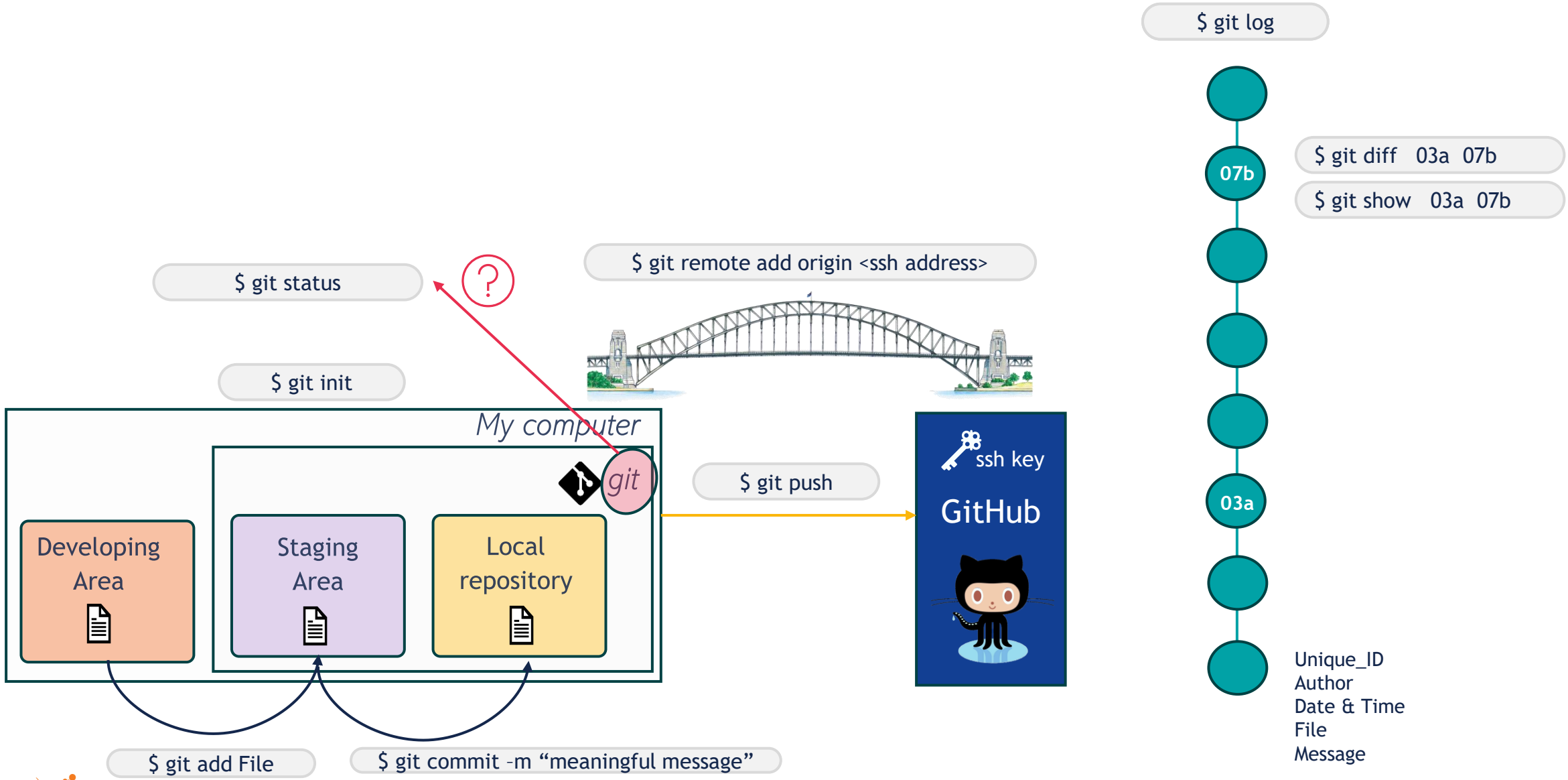


The basic routine to have a version in the local repository is?

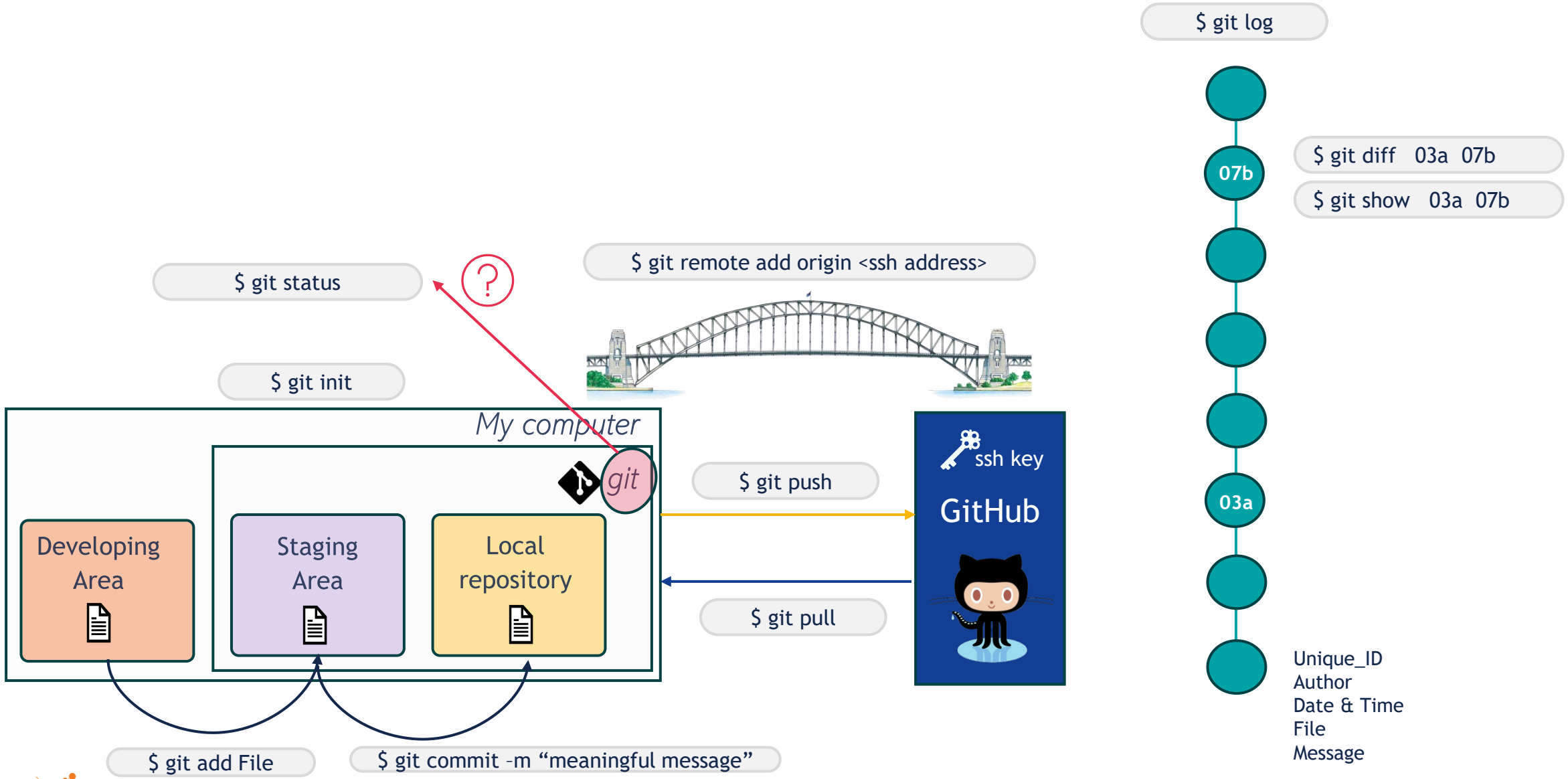




To update my GitHub backup



To sync changes from remote to local

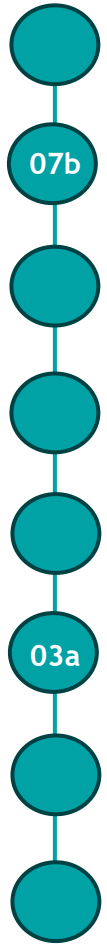


To check my git config

$ git config --global --list



$ git log



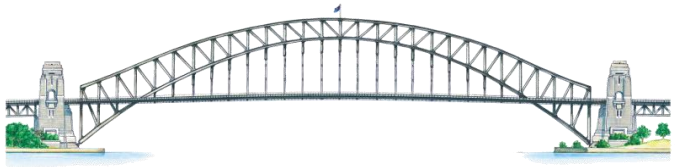
$ git diff 03a 07b

$ git show 03a 07b

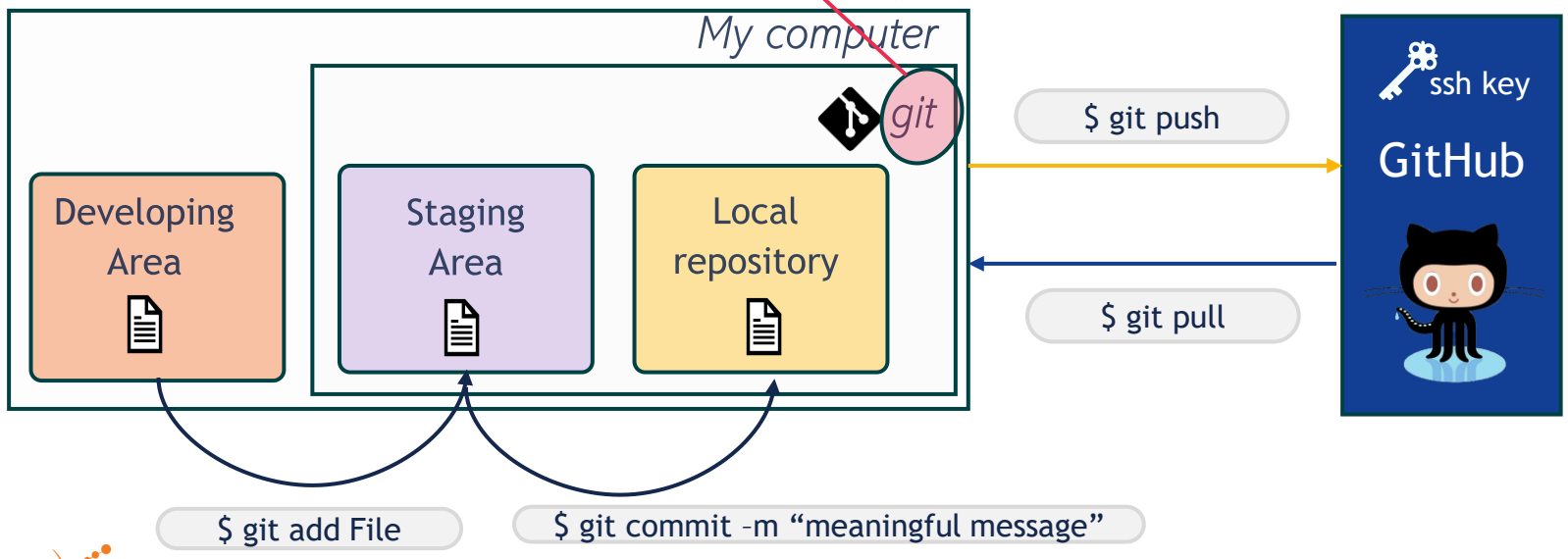
Unique_ID
Author
Date & Time
File
Message

$ git status

$ git remote add origin <ssh address>



$ git init



$ git push

$ git pull

$ git add File

$ git commit -m “meaningful message”

To change my info in the config file

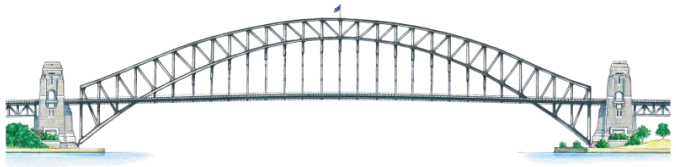
$ git config --global --list

$ git config --global user.name “Your Name”

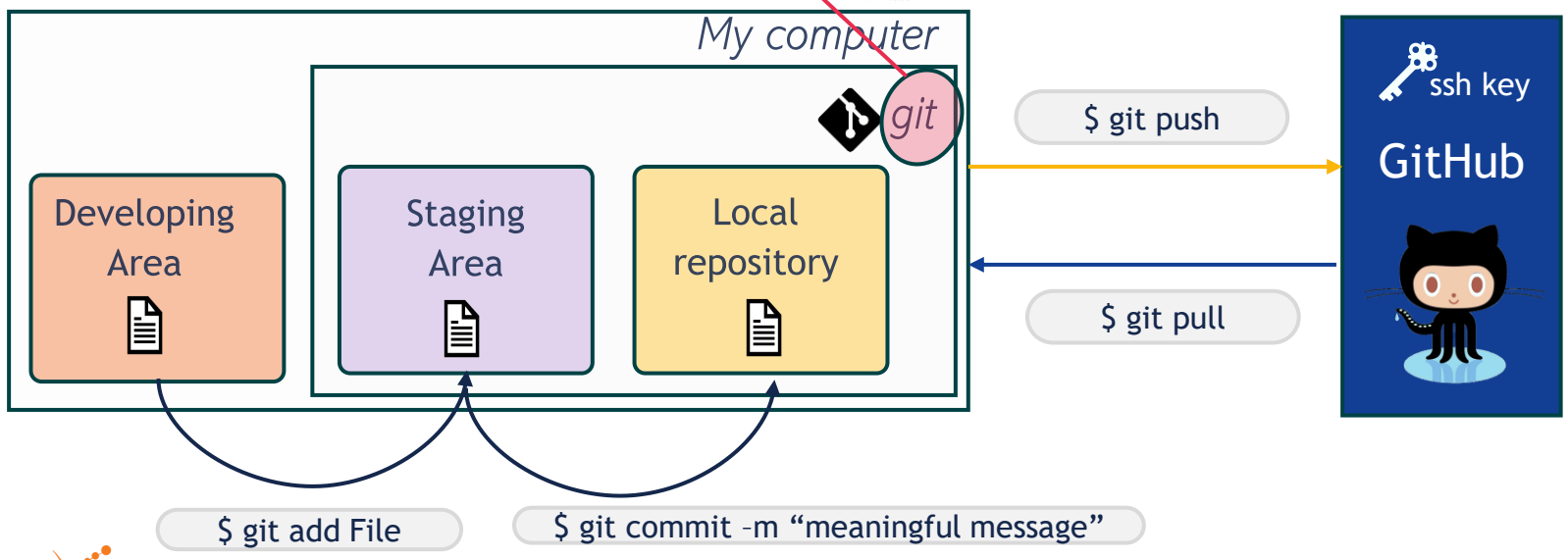
$ git config --global user.email “your_email@domain.com”

$ git status

$ git remote add origin <ssh address>



$ git init



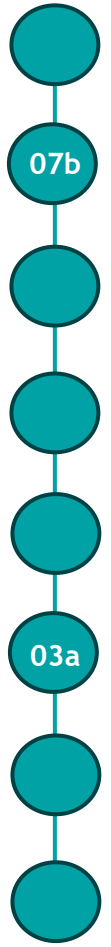
$ git push

$ git pull

$ git add File

$ git commit -m “meaningful message”

$ git log



$ git diff 03a 07b

$ git show 03a 07b

Unique_ID
Author
Date & Time
File
Message



Coffee time


catastrophe!

Catastrophe!

Each person will have a different challenge. You are working in pairs; you should help each other to solve the problem.

First, I need access to your computers !



catastrophe!

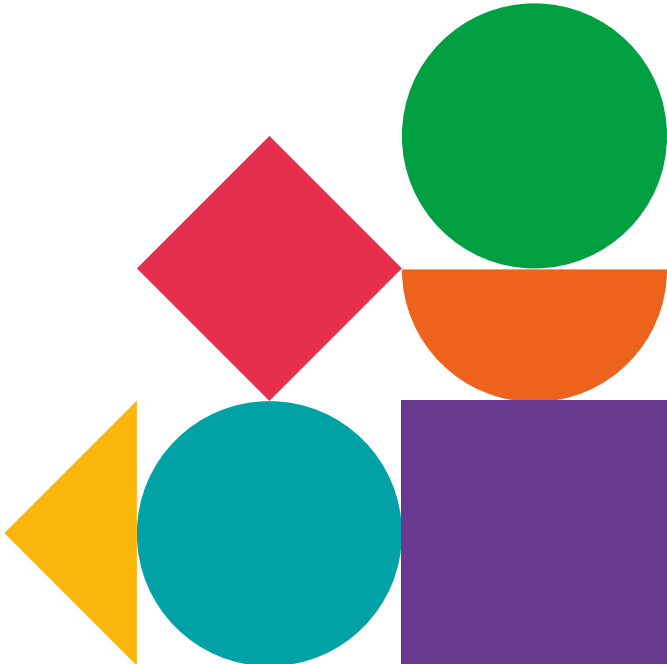
Each person will have a different challenge. You are working in pairs; you should help each other to solve the problem.

First, I need access to your computers !

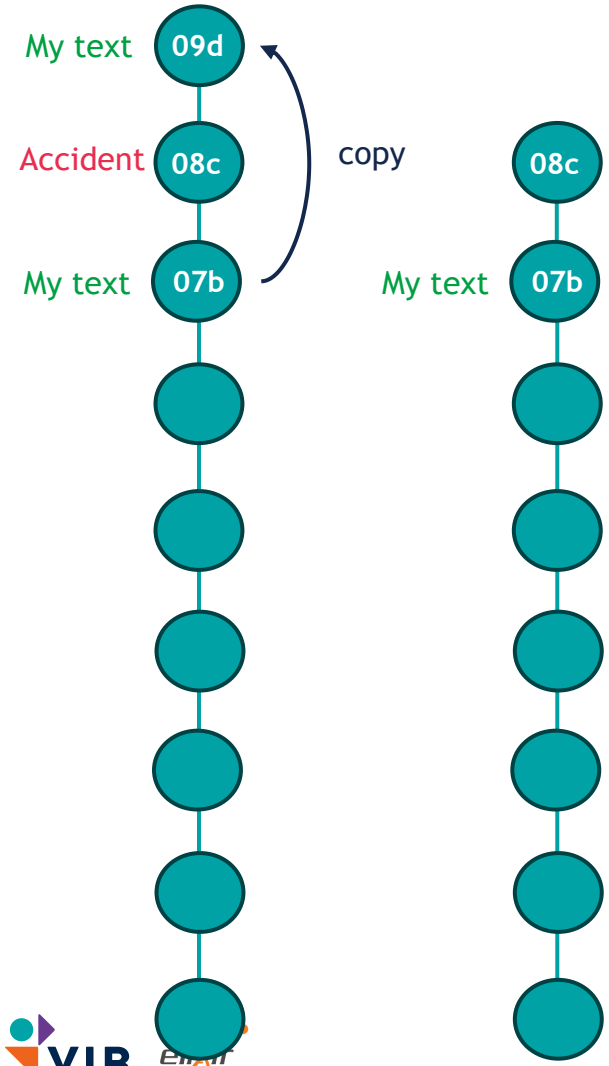
- Challenge A: You messed up!
  - You had a freak commit-accident. Now you need to recover your version to a previous good state.
- Challenge B: You messed up big time!
  - You lost your local repository in a freak deletion accident. You need to recover a full functional version of your local repository. Including your files back.



Challenge A: You had a freak commit-accident



Challenge A: You had a freak commit-accident

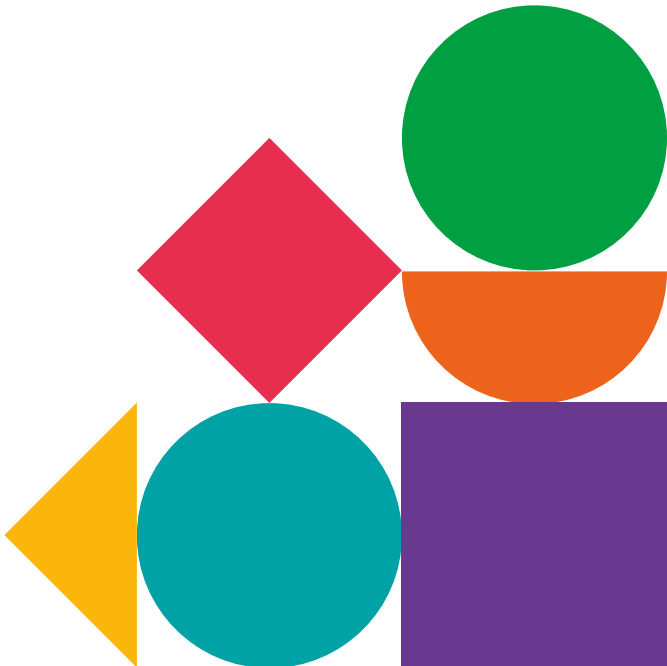


$ git revert

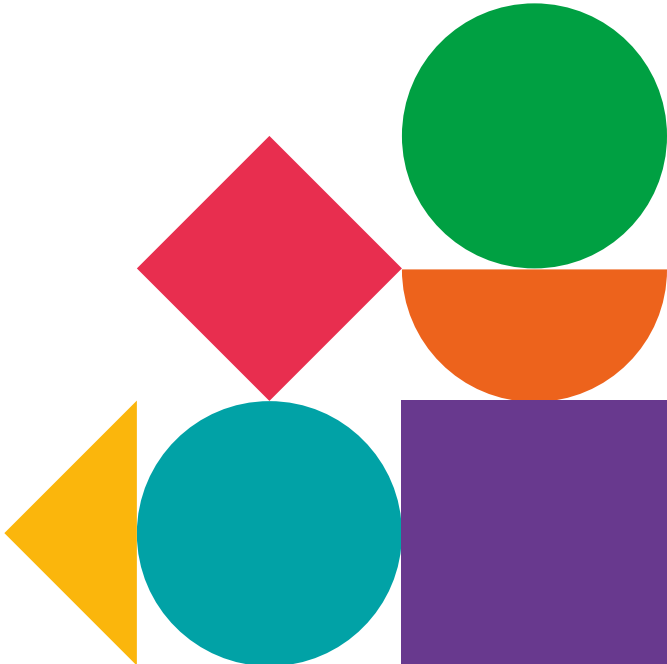
- Copy back the “good” commit
- Keeps in the history
  - committed
  - Reverted

$ git reset

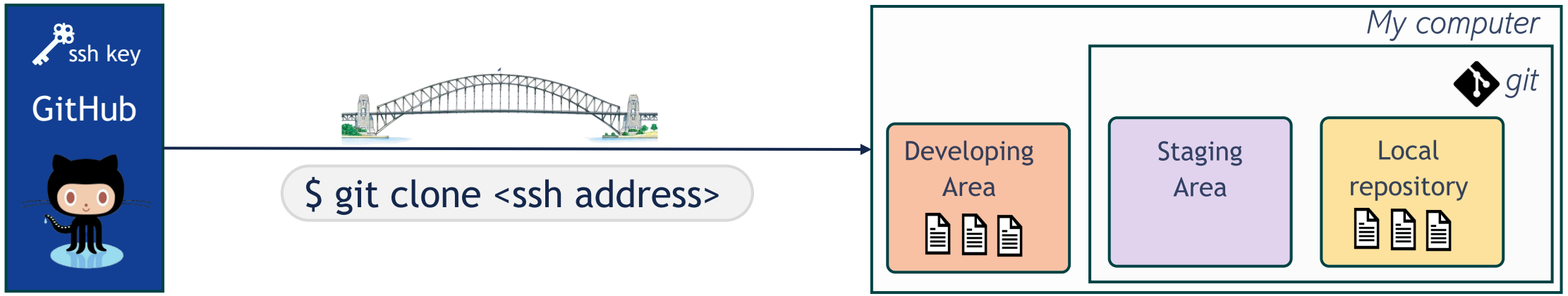
- Dangerous!!
- Discard a commit
- “Unstage” file



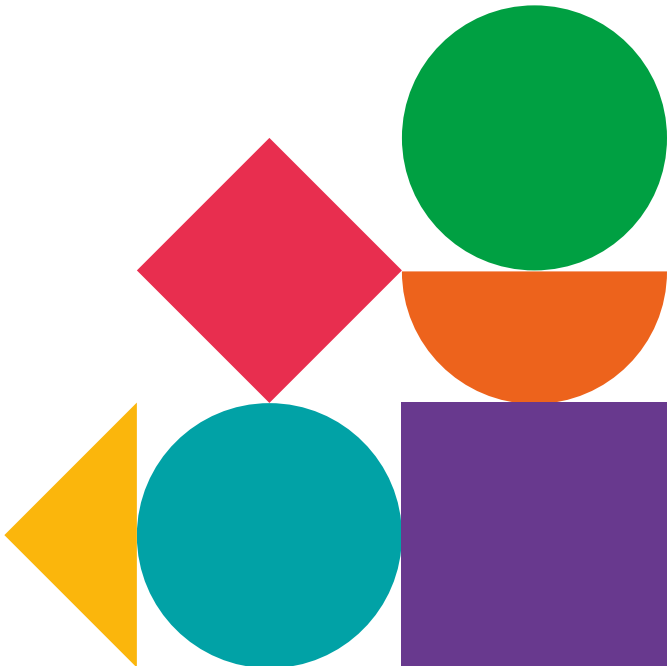
Challenge B: You had a freak deletion accident



Challenge B: You had a freak deletion accident



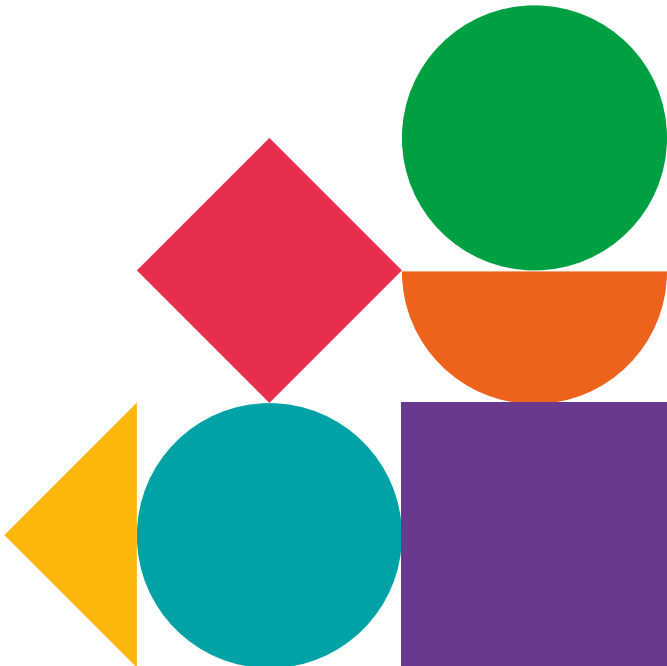
- Downloads repository content (creates dev. area)
- Adds the bridge immediately
- Creates a local repository automatically
- Only works if you have set a ssh key



ACTIVITY

14. Annotate the new commands and concepts I your project.

    - git revert
    - git clone



QUIZ

- What is the difference between `git clone` and `git pull` ?

1. git clone
   - Copies the repo bringing along all the git functionality
     - Local repo
     - Staging area
     - Bridge for sync
2. git pull
   - Updates (sync) the changes from remote to local repositories
   - Can only be used if a local >>> remote repository are connected

QUESTIONS?

